

User Manual

for S32 PWM Driver

Document Number: UM11PWMA SR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	15
3.6 Driver usage and configuration tips	15
3.7 Runtime errors	18
3.8 Symbolic Names Disclaimer	19
4 Tresos Configuration Plug-in	20
4.1 Module Pwm	23
4.2 Container PwmChannelConfigSet	24
4.3 Container PwmChannel	24
4.4 Parameter PwmChannelId	24
4.5 Parameter PwmChannelClass	25
4.6 Parameter PwmPeriodInTicks	26
4.7 Parameter PwmPeriodDefault	26
4.8 Parameter PwmDutycycleDefault	27
4.9 Parameter PwmPolarity	27
4.10 Parameter PwmIdleState	28
4.11 Parameter PwmNotification	28
4.12 Reference PwmChannelEcucPartitionRef	29
4.13 Reference PwmHwChannel	30
4.14 Reference PwmMcuClockReferencePoint	30
4.15 Container PwmFtm	30
4.16 Parameter PwmHwInstance	31
4.17 Container PwmFtmClkCfg	31
4.18 Parameter PwmFtmClockSource	32
4.19 Parameter PwmFtmClockPrescaler	33
4.20 Parameter PwmFtmClockPrescalerAlternate	33
4.21 Container PwmFtmGlobalChCfg	34

4.22 Parameter PwmFtmCounterMode	34
4.23 Parameter PwmFtmPeriod	35
4.24 Parameter PwmFtmPeriodDither	35
4.25 Parameter PwmFtmDeadTime	35
4.26 Parameter PwmFtmDeadTimePrescaler	36
4.27 Container PwmFtmInstCfg	37
4.28 Parameter PwmFtmDebugMode	37
4.29 Parameter PwmFtmWriteProtection	39
4.30 Parameter PwmFtmInitTrigger	39
4.31 Parameter PwmFtmInitTriggerMode	40
4.32 Parameter PwmFtmOverflowIrq	40
4.33 Parameter PwmFtmReloadIrq	41
4.34 Container PwmFtmOverflowIrqCallback	41
4.35 Parameter PwmFtmOverflowIrqFunctionCallback	42
4.36 Parameter PwmFtmOverflowIrqParameterCallback	42
4.37 Container PwmFtmReloadIrqCallback	43
4.38 Parameter PwmFtmReloadIrqFunctionCallback	43
4.39 Parameter PwmFtmReloadIrqParameterCallback	43
4.40 Container PwmFtmSyncCfg	44
4.41 Parameter PwmFtmSyncMode	44
4.42 Parameter PwmFtmHwTrig0	45
4.43 Parameter PwmFtmHwTrig1	45
4.44 Parameter PwmFtmHwTrig2	46
4.45 Parameter PwmFtmAutoHwTrig	46
4.46 Parameter PwmFtmMaxLoadPoint	46
4.47 Parameter PwmFtmMinLoadPoint	47
4.48 Parameter PwmFtmHalfCycleLoadPoint	47
4.49 Parameter PwmFtmHalfCycleLoadPointValue	48
4.50 Parameter PwmFtmReloadPointFrequency	48
4.51 Parameter PwmFtmInvertControl	49
4.52 Parameter PwmFtmSwOutControl	49
4.53 Parameter PwmFtmOutputMask	50
4.54 Parameter PwmFtmCounterInitial	50
4.55 Parameter PwmFtmCounterSync	51
4.56 Container FtmGlobalFaultCfg	51
4.57 Parameter FtmFaultFunctionality	52
4.58 Parameter FtmFaultFilterValue	53
4.59 Parameter FtmFaultOutputState	53
4.60 Parameter FtmFaultIrqUsed	54
4.61 Container FtmFaultSettings	54

4.62 Parameter FtmFaultId	54
4.63 Parameter FtmFaultPolarity	55
4.64 Parameter FtmEnableFaultFilter	55
4.65 Parameter FtmFaultNotification	56
4.66 Container PwmFtmCh	56
4.67 Parameter PwmFtmChId	57
4.68 Parameter PwmFtmChPulseType	57
4.69 Parameter PwmFtmChInitOutput	58
4.70 Parameter PwmFtmChPolarity	58
4.71 Parameter PwmFtmChDutyCycle	59
4.72 Parameter PwmFtmChDutyCycleDither	59
4.73 Parameter PwmFtmChInterrupt	60
4.74 Parameter PwmFtmChPwmOutputEnable	60
4.75 Parameter PwmFtmChSwCtrlEnable	61
4.76 Parameter PwmFtmChExternTriggerEnable	61
4.77 Parameter PwmFtmChSwCtrlValue	62
4.78 Parameter PwmFtmChMatchLoadPointEnable	62
4.79 Parameter PwmFtmPairChEnable	62
4.80 Container PwmFtmChInterruptCallback	63
4.81 Parameter PwmFtmChInterruptFunctionCallback	63
4.82 Parameter PwmFtmChInterruptParameterCallback	64
4.83 Container PwmFtmPairedCh	64
4.84 Parameter PwmFtmPairedChDeadtimeEnable	65
4.85 Parameter PwmFtmPairedChSync	65
4.86 Parameter PwmFtmPairedChExtTrig	66
4.87 Parameter PwmFtmPairedChComplementary	66
4.88 Parameter PwmFtmDutyDitheringPaired	67
4.89 Parameter PwmFtmPairedChCombineMode	67
4.90 Parameter PwmFtmPairedChComplementaryMode	68
4.91 Parameter PwmFtmPairedChPhaseShift	68
4.92 Container PwmGeneral	69
4.93 Parameter PwmMulticoreEnabled	69
4.94 Parameter PwmDevErrorDetect	70
4.95 Parameter PwmDutyCycleUpdatedEndperiod	70
4.96 Parameter PwmPeriodUpdatedEndperiod	71
4.97 Parameter PwmNotificationSupported	71
4.98 Parameter PwmEnableUserModeSupport	72
4.99 Parameter PwmLowPowerStatesSupport	72
4.100 Parameter PwmPowerStateAsynchTransitionMode	73
4.101 Parameter PwmEnableDualClockMode	73

4.102	Parameter PwmEnableExternalTrigger	74
4.103	Parameter PwmEnablePhaseShift	74
4.104	Parameter PwmEnableMaskOutputs	74
4.105	Parameter PwmFaultSupported	75
4.106	Parameter PwmReloadNotificationSupported	75
4.107	Parameter PwmMultiChannelSync	76
4.108	Parameter PwmIndex	76
4.109	Reference PwmEcucPartitionRef	77
4.110	Reference PwmKernelEcucPartitionRef	77
4.111	Container PwmPowerStateConfig	78
4.112	Parameter PwmPowerState	78
4.113	Parameter PwmPowerStateReadyCbKRef	79
4.114	Container PwmConfigurationOfOptApiServices	79
4.115	Parameter PwmDeInitApi	80
4.116	Parameter PwmGetOutputState	80
4.117	Parameter PwmSetDutyCycle	81
4.118	Parameter PwmSetOutputToIdle	81
4.119	Parameter PwmSetPeriodAndDuty	82
4.120	Parameter PwmVersionInfoApi	82
4.121	Parameter PwmGetChannelStateApi	82
4.122	Parameter PwmSetDutyCycle_NoUpdate	83
4.123	Parameter PwmSetPeriodAndDuty_NoUpdate	83
4.124	Parameter PwmSetDutyPhaseShift	85
4.125	Parameter PwmSetChannelDeadTime	85
4.126	Parameter PwmForceOutputToZeroApi	86
4.127	Parameter PwmEnableTrigger	86
4.128	Parameter PwmDisableTrigger	87
4.129	Parameter PwmResetCounter	87
4.130	Container CommonPublishedInformation	88
4.131	Parameter ArReleaseMajorVersion	88
4.132	Parameter ArReleaseMinorVersion	89
4.133	Parameter ArReleaseRevisionVersion	89
4.134	Parameter ModuleId	90
4.135	Parameter SwMajorVersion	90
4.136	Parameter SwMinorVersion	91
4.137	Parameter SwPatchVersion	91
4.138	Parameter VendorApiInfix	92
4.139	Parameter VendorId	92

5 Module Index

94

5.1 Software Specification	94
6 Module Documentation	95
6.1 Ftm Pwm IPL	95
6.1.1 Detailed Description	95
6.1.2 Data Structure Documentation	98
6.1.3 Macro Definition Documentation	101
6.1.4 Types Reference	101
6.1.5 Enum Reference	101
6.1.6 Function Reference	110
6.2 Pwm Driver	122
6.2.1 Detailed Description	122
6.2.2 Data Structure Documentation	127
6.2.3 Macro Definition Documentation	129
6.2.4 Types Reference	146
6.2.5 Enum Reference	146
6.2.6 Function Reference	149



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR PWM for S32. AUTOSAR PWM driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR PWM driver requirements and APIs are described in the AUTOSAR PWM driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of PWM Driver	AUTOSAR Release 4.4.0
2	S32G2 Reference Manual	Rev. 7, February 2023
3	S32G3 Reference Manual	Rev. 3, Feb 2023
4	S32R45 Reference Manual	Rev. 3, 12/2021
5	S32G2 Errata Document	Mask Set Errata for Mask 0P77B Rev. 3.0
6	S32G2 Errata Document	Mask Set Errata for Mask 1P77B Rev. 1.0
7	S32G3 Errata Document	Mask Set Errata for Mask 1P72B Rev. 2.1
8	S32R45 Errata Document	Mask Set Errata for Mask P57D, Rev. 2.0
9	S32G2 Data Sheet	Rev. 6, Dec 2022
10	S32G3 Data Sheet	Rev. 2, RC, Feb 2023
11	VR5510 Data Sheet	Rev. 5, April 2022
12	S32R45 Data Sheet	Rev. 3, RC — 11/2022

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

3.2 Driver Design Summary

The driver provides functions for initialization and control of the microcontroller internal PWM stage (pulse width modulation). The PWM module generates pulses with variable pulse width. It allows the selection of the duty cycle and the signal period time. Each PWM channel is linked to a hardware channel capable of implementing PWM functionality, which belongs to the microcontroller.

The S32 microcontrollers contains two FlexTimer(FTM) module instances.

The FTM module has the following major features in this release:

- FTM source clock is selectable with prescaler option.
- 16-bit counter.
- Each channel can be configured for edge-aligned or center-aligned PWM mode.
- Each pair of channels can be combined to generate a PWM signal with independent control of both edges of PWM signal.
- The FTM channels can operate as pairs with equal outputs, pairs with complementary outputs, or independent channels with independent outputs.
- The deadtime insertion is available for each complementary pair.

3.3 Hardware Resources

Resources available on each derivative.

Derivatives	Pwm Module	Pwm Channels available
s32g274a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g254a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g233a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g234m_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32r45_bga780	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1
s32g399a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g398a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g379a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g378a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g359a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g358a_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g339m_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
s32g338m_bga525	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR PWM Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the PWM Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out

of scope for the PWM driver.

Requirement	Status	Description	Notes
SWS_Pwm_00065	N/S	The Pwm SWS shall not define the code file structure.	Code file structure does always not define in the driver.
SWS_Pwm_70075	N/S	Pwm_Irq.c shall include Pwm.h .	Pwm_Irq.c is not needed. Autosar specific interrupt behaviour is implemented using a normal function placed in the Pwm.c file.
SWS_Pwm_00162	N/S	The Pwm Driver shall support synchronuous and asynchronous power state transitions, depending on the value of the configuration parameter PwmPower↔StateAsynchTransitionMode.	Rejected: All PWM hardware do not support asynchronous power state transitions
SWS_Pwm_00164	N/S	In case the configuration parameter PwmPowerState↔AsynchTransitionMode is set to TRUE, the preparation process shall continue in background after the relative API returns and its completion shall be notified by means of the configured callback.	Rejected: All PWM hardware do not support asynchronous power state transitions
SWS_Pwm_00189	N/S	Service name: - Pwm↔Main_PowerTransition↔Manager - Syntax: - void Pwm_Main_PowerTransitionManager(void) - Service ID[hex]: - 0x0d - Description: - This API is cyclically called and supervises the power state transitions, checking for the readiness of the module and issuing the callbacks IoHwAb_Pwm↔NotifyReadyForPowerState (see PwmPowerStateReadyCbRef configuration parameter). - Available via: - SchM_Pwm.h -	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00190	N/S	This API executes any non-immediate action needed to finalize a power state transition requested by Pwm_PrepatePowerState() .	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00191	N/S	The rate of scheduling shall be defined by Pwm Main↔SchedulePeriod and shall be variable, as the function only needs to be called if a transition has been requested.	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
SWS_Pwm_00192	N/S	This API shall also issue call-back notifications to the eventually registered users (IoHwAbs) as configured, only in case the asynch mode is chosen.	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00193	N/S	In case the PWM module is not initialized, this function shall simply return without any further elaboration. This is needed to avoid to elaborate uninitialized variables. No DET error shall be entered, because this condition can easily be verified during the startup phase (tasks started before the initialization is complete). Rationale: during the startup phase it can happen that the OS already schedules tasks, which call main functions, while some modules are not initialised yet. This is no real error condition, although need handling, i.e. returning without execution. Although the transition state monitoring functionality is mandatory, the implementation of this API is optional, meaning that if the HW allows for other ways to deliver notification and watch the transition state the implementation of this function can be skipped.	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00198	N/S	Service name: - IoHwAb↔ _Pwm_NotifyReadyFor↔ PowerState - Syntax: - void IoHwAb_Pwm_NotifyReady↔ ForPowerState(void) - Service ID[hex]: - 0x60 - Sync/Async: - Synchronous - Reentrancy: - Non Reentrant - Parameters (in): - None - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - The API shall be invoked by the PWM Driver when the requested power state preparation for mode is completed. - Available via: - IoHwAb_Pwm.h -	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
SWS_Pwm_00199	N/S	In case the PWM Driver is configured to support power state management with asynchronous transitions, this API shall be called to signal completion of the power transition preparation phase to the IoHwAbs module. This is a callback, this API is to be implemented in the IoHwAbs component.	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00153	N/S	These requirements are not applicable to this specification.	not a requirement
ECUC_Pwm_00143	N/S	Name - PwmPowerState↔ AsynchTransitionMode - Parent Container - PwmGeneral - Description - Enables / disables support of the PWM Driver to the asynchronous power state transition. - Multiplicity - 0..1 - Type - EcucBooleanParamDef - Default value - false - Post-Build Variant Multiplicity - false - Post-Build Variant Value - false - Multiplicity Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope: localdependency↔ : This parameter shall only be configured if the parameter PwmLowPowerStatesSupport is set to true. -	Rejected: All PWM hardwares do not support asynchronous power state transitions
ECUC_Pwm_00144	N/S	Container Name - PwmPower↔ StateConfig - Description - Each instance of this parameter defines a power state and the callback to be called when this power state is reached. - Configuration Parameters -	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
ECUC_Pwm_00146	N/S	Name - PwmPowerState - Parent Container - PwmPowerStateConfig - Description - Each instance of this parameter describes a different power state supported by the PWM HW. It should be defined by the HW supplier and used by the PWM Driver to reference specific HW configurations which set the PWM HW module in the referenced power state. At least the power mode corresponding to full power state shall be always configured. - Multiplicity - 1 - Type - EcucIntegerParamDef (Symbolic Name generated for this parameter) - Range - 0 .. 18446744073709551615 - Default value - - - Post-Build Variant Value - false - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope: localdependency: This parameter shall only be configured if the parameter PwmLowPowerStatesSupport is set to true. -	Rejected: All PWM hardwares do not support asynchronous power state transitions
ECUC_Pwm_00145	N/S	Name - PwmPowerStateReadyCbRef - Parent Container - PwmPowerStateConfig - Description - Each instance of this parameter contains a reference to a power mode callback defined in a CDD or IoHwAbs component. - Multiplicity - 1 - Type - EcucFunctionNameDef - Default value - - - maxLength - - - minLength - - - regularExpression - - - Post-Build Variant Value - false - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope: localdependency: This parameter shall only be configured if the parameter PwmLowPowerStatesSupport is set to true. -	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
SWS_Pwm_CONSTR_00001	N/S	DRAFT: The ECUC partitions referenced by PwmKernel↔EcucPartitionRef shall be a subset of the ECUC partitions referenced by PwmEcuc↔PartitionRef.	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Pwm↔KernelEcucPartitionRef is not supported
SWS_Pwm_91003	N/S	Service name: - Pwm↔_DisableNotification (draft) - Syntax: - void Pwm_DisableNotification(Pwm_ChannelType ChannelNumber) - Service ID[hex]: - 0x06 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant for different channel numbers - Parameters (in): - Channel↔Number - Numeric identifier of the PWM - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Service to disable the PWM signal edge notification.Tags: atp.↔Status=draft - Available via: - Pwm.h -	Description specified as draft is not clear. Should be re-assessed on next ASR version
SWS_Pwm_91004	N/S	Service name: - Pwm↔_EnableNotification (draft) - Syntax: - void Pwm_EnableNotification(Pwm_ChannelType ChannelNumber, Pwm_EdgeNot - Service ID[hex]: - 0x07 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant for different channel numbers - Parameters (in): - ChannelNumber - Numeric identifier of the PWM - Notification - Type of notification PWM_RISING_EDGE or PWM_FALLING_EDGE or PWM_BOTH_EDGES - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Service to enable the PWM signal edge notification according to notification parameter.Tags: atp.Status=draft - Available via: - Pwm.h -	Description specified as draft is not clear. Should be re-assessed on next ASR version

Requirement	Status	Description	Notes
SWS_Pwm_91000	N/S	Service name: - Pwm↔ _SetDutyCycle (draft) - Syntax: - void Pwm_SetDutyCycle(Pwm_ChannelType ChannelNumber, uint16 DutyCycle) - Service ID[hex]: - 0x02 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant for different channel numbers - Parameters (in): - Channel↔ Number - Numeric identifier of the PWM - DutyCycle - Min=0x0000 Max=0x8000 - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Service sets the duty cycle of the PWM channel.Tags: atp.Status=draft - Available via: - Pwm.h -	Description specified as draft is not clear. Should be re-assessed on next ASR version
SWS_Pwm_91002	N/S	Service name: - Pwm↔ SetOutputToIdle (draft) - Syntax: - void Pwm_SetOutputToIdle(Pwm_ChannelType ChannelNumber) - Service ID[hex]: - 0x04 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant for different channel numbers - Parameters (in): - Channel↔ Number - Numeric identifier of the PWM - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Service sets the PWM output to the configured Idle state.Tags: atp.Status=draft - Available via: - Pwm.h -	Description specified as draft is not clear. Should be re-assessed on next ASR version

Requirement	Status	Description	Notes
SWS_Pwm_91001	N/S	Service name: - Pwm↵ _SetPeriodAndDuty (draft) - Syntax: - void Pwm_SetPeriodAndDuty(Pwm_ChannelType ChannelNumber, Pwm_PeriodType - Service ID[hex]: - 0x03 - Sync/Async: - Asynchronous - Reentrancy: - Reentrant for different channel numbers - Parameters (in): - Channel↵ Number - Numeric identifier of the PWM - Period - Period of the PWM signal - DutyCycle - Min=0x0000 Max=0x8000 - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - Service sets the period and the duty cycle of a PWM channelTags: atp.↵ Status=draft - Available via: - Pwm.h -	Description specified as draft is not clear. Should be re-assessed on next ASR version
ECUC_Pwm_00150	N/S	Name - PwmKernelEcuc↵ PartitionRef - Parent Container - PwmGeneral - Description - Maps the PWM kernel to zero or one ECUC partitions to assign the driver kernel to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the PWM driver is mapped to.Tags: atp.Status=draft - Multiplicity - 0..1 - Type - Reference to [EcucPartition] - Post-Build Variant Multiplicity - true - Post-Build Variant Value - true - Multiplicity Con- figuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Value Configuration Class - Pre-compile time - X - All Variants - Link time - - - - Post-build time - - - - Scope / Dependency - scope: ECU -	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Pwm↵ KernelEcucPartitionRef is not supported
SWS_Pwm_00175	N/S	The API shall report the DET error PWM_E_TRANSITIO↵ N_NOT_POSSIBLE in case the requested power state can- not be directly reached from the current power state.	Currently, Pwm driver has only 2 powe state (High power and Low Power) so this DET error will not happen.

Requirement	Status	Description	Notes
SWS_Pwm_00195	N/S	The API shall report the DET error PWM_E_TRANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state.	Currently, Pwm driver has only 2 power state (High power and Low Power) so this DET error will not happen.
SWS_Pwm_00188	N/S	The API shall report the DET error PWM_E_TRANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state. All asynchronous operation needed to reach the target power state can be executed in background in the context of Pwm_Main_PowerTransitionManager.	Currently, Pwm driver has only 2 power state (High power and Low Power) so this DET error will not happen.

3.5 Driver Limitations

None.

3.6 Driver usage and configuration tips

How to configure in EB and S32DS:

Basically, PWM driver should be configured by EB tresos or S32DS in 3 steps:

Step 1: Configure FTM instances in PwmFtm tab

-Example with EB tresos

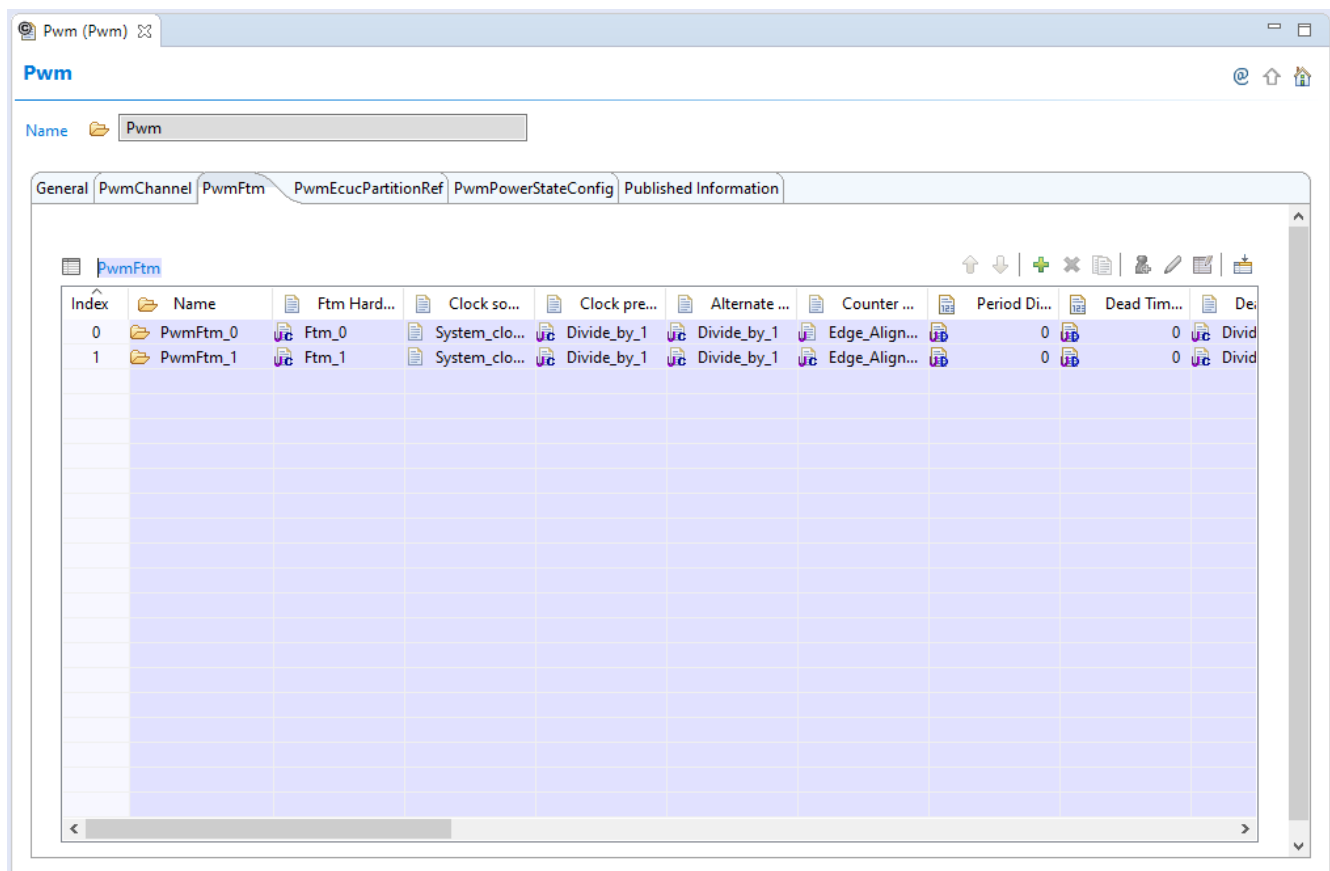


Figure 3.1 Configure FTM instances

Step 2: Configure FTM channels in PwmFtmCh tab

-Example with EB tresos

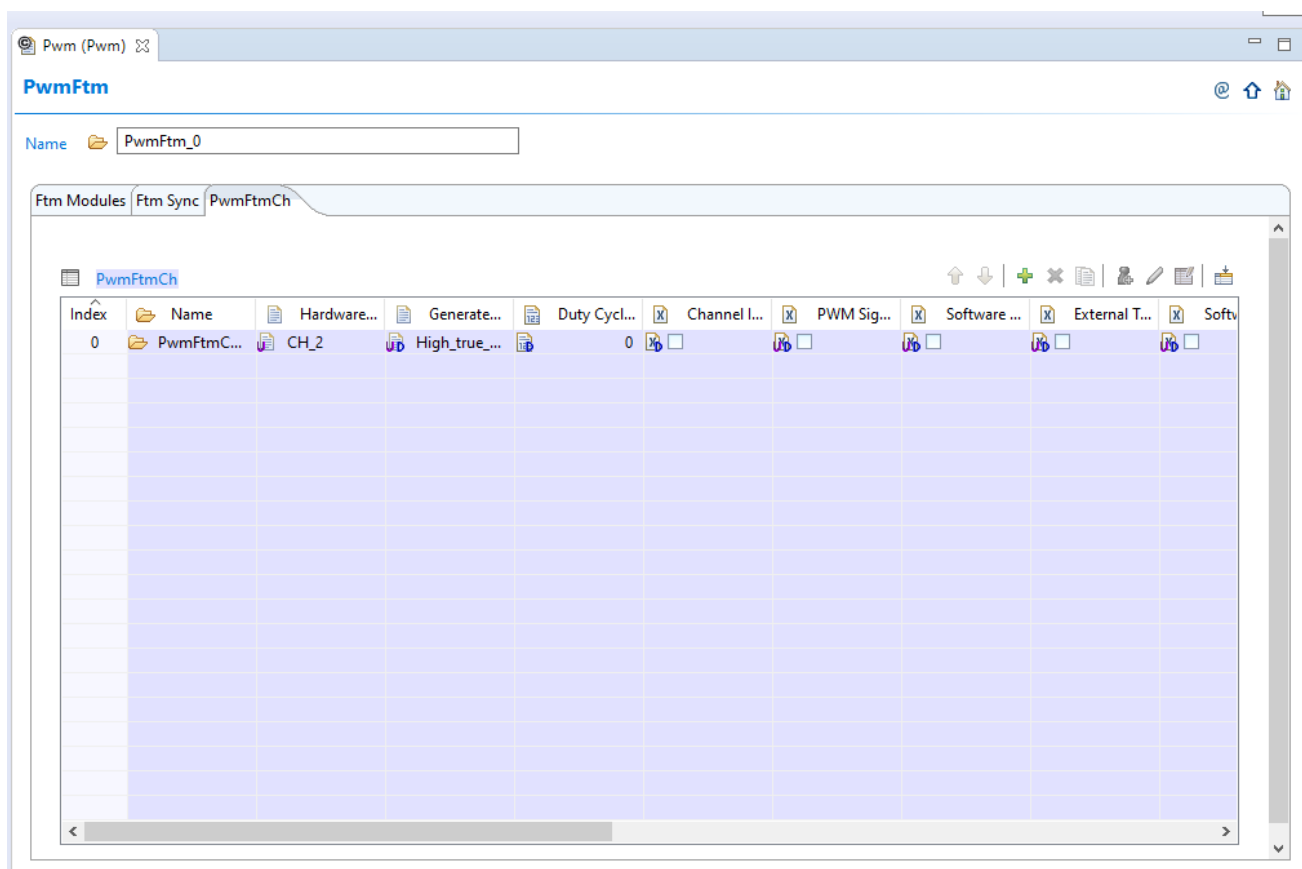


Figure 3.2 Configure FTM channels

Step 3: Configure logic channels in PwmChannel tab

-Example with EB tresos

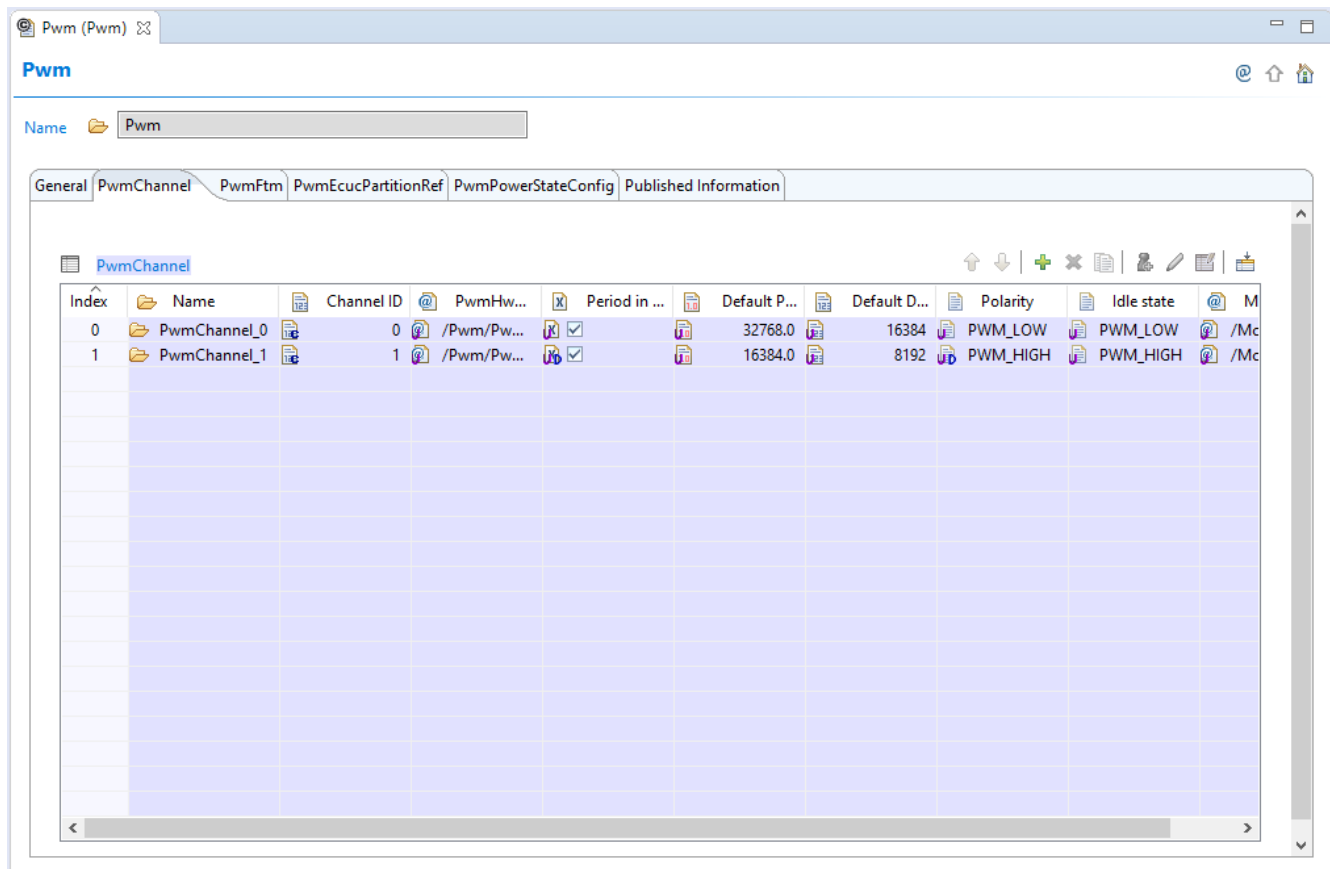


Figure 3.3 Configure logic channels

Note when using interrupt functions in the IPV layer:

To make it more convenient to use the 2 enable/disable interrupt functions when using the IPV layer, we have updated parameters of functions (Refer to Ftm_Pwm_Ip_NotifType). So when using these 2 functions you need to understand the meaning of the input parameters, namely:

- From FTM_PWM_IP_CHANNEL_0_NOTIFICATION to FTM_PWM_IP_CHANNEL_7_NOTIFICATION - Used for channel interrupts handling.
- FTM_PWM_IP_OVERFLOW_NOTIFICATION - Used for overflow interrupts handling
- FTM_PWM_IP_RELOAD_POINT_NOTIFICATION - Used for reload interrupts handling
- FTM_PWM_IP_FAULT_NOTIFICATION - Used for fault interrupts handling

3.7 Runtime errors

None.

3.8 Symbolic Names Disclaimer

All containers having `symbolicNameValue` set to `TRUE` in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Pwm](#)
 - Container [PwmChannelConfigSet](#)
 - * Container [PwmChannel](#)
 - Parameter [PwmChannelId](#)
 - Parameter [PwmChannelClass](#)
 - Parameter [PwmPeriodInTicks](#)
 - Parameter [PwmPeriodDefault](#)
 - Parameter [PwmDutycycleDefault](#)
 - Parameter [PwmPolarity](#)
 - Parameter [PwmIdleState](#)
 - Parameter [PwmNotification](#)
 - Reference [PwmChannelEcucPartitionRef](#)
 - Reference [PwmHwChannel](#)
 - Reference [PwmMcuClockReferencePoint](#)
 - * Container [PwmFtm](#)
 - Parameter [PwmHwInstance](#)
 - Container [PwmFtmClkCfg](#)
 - Parameter [PwmFtmClockSource](#)
 - Parameter [PwmFtmClockPrescaler](#)
 - Parameter [PwmFtmClockPrescalerAlternate](#)
 - Container [PwmFtmGlobalChCfg](#)
 - Parameter [PwmFtmCounterMode](#)
 - Parameter [PwmFtmPeriod](#)
 - Parameter [PwmFtmPeriodDither](#)
 - Parameter [PwmFtmDeadTime](#)
 - Parameter [PwmFtmDeadTimePrescaler](#)
 - Container [PwmFtmInstCfg](#)
 - Parameter [PwmFtmDebugMode](#)
 - Parameter [PwmFtmWriteProtection](#)
 - Parameter [PwmFtmInitTrigger](#)

- Parameter [PwmFtmInitTriggerMode](#)
- Parameter [PwmFtmOverflowIrq](#)
- Parameter [PwmFtmReloadIrq](#)
- Container [PwmFtmOverflowIrqCallback](#)
- Parameter [PwmFtmOverflowIrqFunctionCallback](#)
- Parameter [PwmFtmOverflowIrqParameterCallback](#)
- Container [PwmFtmReloadIrqCallback](#)
- Parameter [PwmFtmReloadIrqFunctionCallback](#)
- Parameter [PwmFtmReloadIrqParameterCallback](#)
- Container [PwmFtmSyncCfg](#)
- Parameter [PwmFtmSyncMode](#)
- Parameter [PwmFtmHwTrig0](#)
- Parameter [PwmFtmHwTrig1](#)
- Parameter [PwmFtmHwTrig2](#)
- Parameter [PwmFtmAutoHwTrig](#)
- Parameter [PwmFtmMaxLoadPoint](#)
- Parameter [PwmFtmMinLoadPoint](#)
- Parameter [PwmFtmHalfCycleLoadPoint](#)
- Parameter [PwmFtmHalfCycleLoadPointValue](#)
- Parameter [PwmFtmReloadPointFrequency](#)
- Parameter [PwmFtmInvertControl](#)
- Parameter [PwmFtmSwOutControl](#)
- Parameter [PwmFtmOutputMask](#)
- Parameter [PwmFtmCounterInitial](#)
- Parameter [PwmFtmCounterSync](#)
- Container [FtmGlobalFaultCfg](#)
- Parameter [FtmFaultFunctionality](#)
- Parameter [FtmFaultFilterValue](#)
- Parameter [FtmFaultOutputState](#)
- Parameter [FtmFaultIrqUsed](#)
- Container [FtmFaultSettings](#)
- Parameter [FtmFaultId](#)
- Parameter [FtmFaultPolarity](#)
- Parameter [FtmEnableFaultFilter](#)
- Parameter [FtmFaultNotification](#)
- Container [PwmFtmCh](#)
- Parameter [PwmFtmChId](#)
- Parameter [PwmFtmChPulseType](#)
- Parameter [PwmFtmChInitOutput](#)
- Parameter [PwmFtmChPolarity](#)
- Parameter [PwmFtmChDutyCycle](#)
- Parameter [PwmFtmChDutyCycleDither](#)
- Parameter [PwmFtmChInterrupt](#)
- Parameter [PwmFtmChPwmOutputEnable](#)
- Parameter [PwmFtmChSwCtrlEnable](#)
- Parameter [PwmFtmChExternTriggerEnable](#)
- Parameter [PwmFtmChSwCtrlValue](#)

- Parameter [PwmFtmChMatchLoadPointEnable](#)
- Parameter [PwmFtmPairChEnable](#)
- Container [PwmFtmChInterruptCallback](#)
- Parameter [PwmFtmChInterruptFunctionCallback](#)
- Parameter [PwmFtmChInterruptParameterCallback](#)
- Container [PwmFtmPairedCh](#)
- Parameter [PwmFtmPairedChDeadtimeEnable](#)
- Parameter [PwmFtmPairedChSync](#)
- Parameter [PwmFtmPairedChExtTrig](#)
- Parameter [PwmFtmPairedChComplementary](#)
- Parameter [PwmFtmDutyDitheringPaired](#)
- Parameter [PwmFtmPairedChCombineMode](#)
- Parameter [PwmFtmPairedChComplementaryMode](#)
- Parameter [PwmFtmPairedChPhaseShift](#)
- Container [PwmGeneral](#)
 - * Parameter [PwmMulticoreEnabled](#)
 - * Parameter [PwmDevErrorDetect](#)
 - * Parameter [PwmDutycycleUpdatedEndperiod](#)
 - * Parameter [PwmPeriodUpdatedEndperiod](#)
 - * Parameter [PwmNotificationSupported](#)
 - * Parameter [PwmEnableUserModeSupport](#)
 - * Parameter [PwmLowPowerStatesSupport](#)
 - * Parameter [PwmPowerStateAsynchTransitionMode](#)
 - * Parameter [PwmEnableDualClockMode](#)
 - * Parameter [PwmEnableExternalTrigger](#)
 - * Parameter [PwmEnablePhaseShift](#)
 - * Parameter [PwmEnableMaskOutputs](#)
 - * Parameter [PwmFaultSupported](#)
 - * Parameter [PwmReloadNotificationSupported](#)
 - * Parameter [PwmMultiChannelSync](#)
 - * Parameter [PwmIndex](#)
 - * Reference [PwmEcucPartitionRef](#)
 - * Reference [PwmKernelEcucPartitionRef](#)
 - * Container [PwmPowerStateConfig](#)
 - Parameter [PwmPowerState](#)
 - Parameter [PwmPowerStateReadyCbRef](#)
- Container [PwmConfigurationOfOptApiServices](#)
 - * Parameter [PwmDeInitApi](#)
 - * Parameter [PwmGetOutputState](#)
 - * Parameter [PwmSetDutyCycle](#)
 - * Parameter [PwmSetOutputToIdle](#)
 - * Parameter [PwmSetPeriodAndDuty](#)

- * Parameter [PwmVersionInfoApi](#)
- * Parameter [PwmGetChannelStateApi](#)
- * Parameter [PwmSetDutyCycle_NoUpdate](#)
- * Parameter [PwmSetPeriodAndDuty_NoUpdate](#)
- * Parameter [PwmSetDutyPhaseShift](#)
- * Parameter [PwmSetChannelDeadTime](#)
- * Parameter [PwmForceOutputToZeroApi](#)
- * Parameter [PwmEnableTrigger](#)
- * Parameter [PwmDisableTrigger](#)
- * Parameter [PwmResetCounter](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1 Module Pwm

Configuration of Pwm (Pulse Width Modulation) module.

Included containers:

- [PwmChannelConfigSet](#)
- [PwmGeneral](#)
- [PwmConfigurationOfOptApiServices](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container PwmChannelConfigSet

Container contains the channel configuration parameter of the Pwm driver.

Included subcontainers:

- [PwmChannel](#)
- [PwmFtm](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Container PwmChannel

Configuration of an individual Pwm channel.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.4 Parameter PwmChannelId

Channel ID of the Pwm channel. This value will be assigned to the symbolic name derived of the PwmChannel container short name.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.5 Parameter PwmChannelClass

Class of Pwm Channel.

PWM_FIXED_PERIOD - Period of the channel will not be changed.

PWM_FIXED_PERIOD_SHIFTED - Period of the channel will not be changed, and support with phase shift feature.

PWM_VARIABLE_PERIOD - Period of the channel can be changed.

<note>

Due to Ftm hardware specific feature that the counter register and period register are shared common for all channels in one Ftm instance, therefore if current channel is configured PWM_FIXED_PERIOD class, but another channel in the same Ftm instance is configured PWM_VARIABLE_PERIOD, then when that channel call function to

change period, current channel will change period, too.

</note>

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD

4.6 Parameter PwmPeriodInTicks

Check this option to configure Default Period unit in ticks, or uncheck this to configure Default Period unit in seconds.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.7 Parameter PwmPeriodDefault

Default period value of Pwm channel at initialization.

The measure unit are in ticks (if Period In Ticks checked), or in seconds (if unchecked)

Valid range: [0, 65534]

<note>

The maximum period tick is 65534 (instead of maximum value of period register 65535) in order to achieve perfect 0 or 100% duty cycle.

All channels which are in the same Ftm instance with current channel must have the same Default Period value, due to Ftm hardware specific feature that the period register is shared common for all channels in one Ftm instance.

</note>

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1.0
max	65534.0
min	0.0

4.8 Parameter PwmDutycycleDefault

Default value for duty cycle of Pwm channel at initialization.

0 represents for 0% duty cycle

16384 (0x4000) represents for 50% duty cycle

32768 (0x8000) represents for 100% duty cycle

Valid value: [0,32768]

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	16384
max	32768
min	0

4.9 Parameter PwmPolarity

Define the polarity of Pwm channel at initialization.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	PWM_HIGH
literals	['PWM_HIGH', 'PWM_LOW']

4.10 Parameter PwmIdleState

Define Pwm channel state when the output is set to idle.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	PWM_LOW
literals	['PWM_HIGH', 'PWM_LOW']

4.11 Parameter PwmNotification

User callback notification function.

This option is only activated when PwmGeneral/PwmNotificationSupported is checked.

<note>

Use NULL_PTR without any quotes to determine no notification function is used.

If the string is different from above, it will be used as the notification function name.

Notification does not apply to channel that its alignment type is PWM_CENTER_ALIGNED.

</note>

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.12 Reference PwmChannelEcucPartitionRef

Maps a Pwm channel to zero or multiple ECUC partitions to limit the access to this channel group.

The ECUC partitions referenced are a subset of the ECUC partitions where the Pwm driver is mapped to.

When users choose ENABLE multicore feature by checking PwmMulticoreEnabled option, this will force to configure at least 1 ECUC partition in this list that is referenced from ECUC module; OR when DISABLE multicore feature, user have to remove all ECUC partitions in this list.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.13 Reference PwmHwChannel

Select the hw channel on which the functionality of the current PWM channel will be implemented.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D11M40I2R0/Pwm/PwmChannelConfigSet/PwmFtm/PwmFtmCh']

4.14 Reference PwmMcuClockReferencePoint

Reference to the clock source configuration, which is set in the MCU driver configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.15 Container PwmFtm

Configuration of a Ftm module available on the platform.

Included subcontainers:

- [PwmFtmClkCfg](#)
- [PwmFtmGlobalChCfg](#)
- [PwmFtmInstCfg](#)
- [PwmFtmSyncCfg](#)
- [FtmGlobalFaultCfg](#)
- [PwmFtmCh](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.16 Parameter PwmHwInstance

Select the Ftm hardware module instance available on the current platform.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	Ftm_0
literals	['Ftm_0', 'Ftm_1']

4.17 Container PwmFtmClkCfg

Ftm Clock configuration settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.18 Parameter PwmFtmClockSource

Select Ftm module clock source.

This option will be written into Clock Source Selection bitfield (CLKS) of Status and Control register (SC).

FTM_PWM_IP_CLOCK_SOURCE_NONE : No input clock is given to Ftm. Ftm counter is not operational.

FTM_PWM_IP_CLOCK_SOURCE_SYSTEMCLK : System clock is given to Ftm, which is referred to PER_CLK.

FTM_PWM_IP_CLOCK_SOURCE_FIXEDCLK : An fixed frequency clock is given to Ftm. This selection is not supported in current platform.

FTM_PWM_IP_CLOCK_SOURCE_EXTERNALCLK : An external clock is given to Ftm, which is referred to Ftm_x_REF_CLK (x = 0 or 1).

<note>

Clock source and frequency are dependent on the value of the clock that is configured in MCU

and referred by MCU clock reference. Make sure that the correct reference is used for the configured clock.

</note>

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	System_clock
literals	['No_clock', 'System_clock', 'Fixed_clock', 'External_clock']

4.19 Parameter PwmFtmClockPrescaler

Select the prescaler value for clock source.

This option will be written into Prescale Factor Selection bitfield (PS) of Status and Control register (SC).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Divide_by_1
literals	['Divide_by_1', 'Divide_by_2', 'Divide_by_4', 'Divide_by_8', 'Divide_by_16', 'Divide_by_32', 'Divide_by_64', 'Divide_by_128']

4.20 Parameter PwmFtmClockPrescalerAlternate

Select the prescaler value for clock source.

This option will be written into Prescale Factor Selection bitfield (PS) of Status and Control register (SC).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Divide_by_1
literals	['Divide_by_1', 'Divide_by_2', 'Divide_by_4', 'Divide_by_8', 'Divide_by_16', 'Divide_by_32', 'Divide_by_64', 'Divide_by_128']

4.21 Container PwmFtmGlobalChCfg

Ftm Clock configuration settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.22 Parameter PwmFtmCounterMode

Select edge alignment type which is applied to all channels in current Ftm module.

This option will be written into Center-Aligned Pwm Select (CPWMS) of Status and Control register (SC).

FTM_PWM_IP_EDGE_ALIGNED : The leading edges of all Pwm signals are aligned with the beginning of the period.

Ftm counter operates in Up Counting mode.

FTM_PWM_IP_CENTER_ALIGNED : The pulse width centers for all Pwm channels are aligned with the value of CNTIN.

Ftm counter operates in Up-Down Counting mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	Edge_Aligned_mode
literals	['Edge_Aligned_mode', 'Center_Aligned_mode']

4.23 Parameter PwmFtmPeriod

User need to check the FTM frequency before enter the period in frequency of PWM. The min pwm frequency must be greater than (FTM counter frequency / 65536).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	65535
min	0

4.24 Parameter PwmFtmPeriodDither

The value of the dithering applied to the PWM frequency.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.25 Parameter PwmFtmDeadTime

Deadtime counter used to create a offset between the output of two consecutive Ftm channels used complementary

Tresos Configuration Plug-in

modes.

Complementary channels behave as a single channel with two outputs which may have a deadtime between them.

This option will be written into Extended Deadtime Value bitfield (DTVALEX) and Deadtime Value bitfield (DTVAL)

in Deadtime Configuration register (DEADTIME).

Valid value: [0,1023]

<note>For channels which are configured as independent this value has no effect.</note>

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	1023
min	0

4.26 Parameter PwmFtmDeadTimePrescaler

Deadtime prescaler.

This option will be written into Deadtime Prescaler Value (DTPS) in Deadtime Configuration register (DEADTIME).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Divide_by_1
literals	'Divide_by_1', 'Divide_by_4', 'Divide_by_16', 'Divide_by_64', 'Divide_by_256', 'Divide_by_1024'

4.27 Container PwmFtmInstCfg

Ftm Instance configuration settings.

Included subcontainers:

- [PwmFtmOverflowIrqCallback](#)
- [PwmFtmReloadIrqCallback](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.28 Parameter PwmFtmDebugMode

Select the Ftm behavior in Debug mode. Refer to Debug mode sub-chapter for more details.

This option will be written into Debug Mode (BDMMODE) in Configuration register (CONF).

Mode 00

Counter - Stopped

Flag bit - Can be set

Output - Normal

Bufferd Registers - Writes bypass the registers buffers

Mode 01

Counter - Stopped

Flag bit - Is not set

Output - Forced to their safe value according to POLn bit

Bufferd Registers - Writes bypass the registers buffers

Mode 10

Counter - Stopped

Tresos Configuration Plug-in

Flag bit - Is not set

Output - Frozen when the chip enters Debug mode

Bufferd Registers - Writes bypass the registers buffers

Mode 11

Counter - Running

Flag bit - Can be set

Output - Normal

Bufferd Registers - Normal

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Debug_Mode_00
literals	['Debug_Mode_00', 'Debug_Mode_01', 'Debug_Mode_10', 'Debug_Mode_11']

4.29 Parameter PwmFtmWriteProtection

Enable/Disable the write protection of FTM register/bitfields.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.30 Parameter PwmFtmInitTrigger

Enable/Disable the generation of an initialization trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.31 Parameter PwmFtmInitTriggerMode

Select the generation of initialization trigger mode.

Trigger on Reload Point - Initialization trigger is generated when FTM counter reaches a reload point.

Trigger on Counter Update - Initialization trigger is generated when FTM counter is updated.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Trigger_on_Reload_Point
literals	['Trigger_on_Reload_Point', 'Trigger_on_Counter_Update']

4.32 Parameter PwmFtmOverflowIrq

Enable/Disable the generation of an initialization trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.33 Parameter PwmFtmReloadIrq

Enable/Disable the generation of an initialization trigger.

Do not support for this implementation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.34 Container PwmFtmOverflowIrqCallback

Configure Notification function and parameter for interrupt handler callback.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.35 Parameter PwmFtmOverflowIrqFunctionCallback

User callback function.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.36 Parameter PwmFtmOverflowIrqParameterCallback

User callback parameter.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured parameter name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.37 Container PwmFtmReloadIrqCallback

Configure Notification function and parameter for interrupt handler callback.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.38 Parameter PwmFtmReloadIrqFunctionCallback

User callback function.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.39 Parameter PwmFtmReloadIrqParameterCallback

User callback parameter.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured parameter name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.40 Container PwmFtmSyncCfg

Ftm Synchronization configuration settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.41 Parameter PwmFtmSyncMode

Configure FTM Sync to use Software or Hardware trigger modes

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Software_sync_trigger
literals	['Software_sync_trigger', 'Hardware_sync_trigger']

4.42 Parameter PwmFtmHwTrig0

Enables/disables hardware trigger for register sync

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.43 Parameter PwmFtmHwTrig1

Enables/disables hardware trigger for register sync

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.44 Parameter PwmFtmHwTrig2

Enables/disables hardware trigger for register sync

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.45 Parameter PwmFtmAutoHwTrig

Enable/disable auto clear of triggers in hardware trigger mode

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.46 Parameter PwmFtmMaxLoadPoint

Enable/disable max counter value loading point

(for Center-Aligned Pwm, the Reload point is in middle of period)

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.47 Parameter PwmFtmMinLoadPoint

Enable/disable min counter value loading point.

(for Center-Aligned Pwm, the Reload point is in end of period)

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.48 Parameter PwmFtmHalfCycleLoadPoint

Enable/disable half cycle loading point

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.49 Parameter PwmFtmHalfCycleLoadPointValue

This option will be written into Half Cycle register (HCR).

After FTM counter reaches this value, a reload opportunity is generated if bit HCSEL in PWMLOAD register is enabled.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65534
min	0

4.50 Parameter PwmFtmReloadPointFrequency

Select the number of enabled reload opportunities should happen until an enabled reload opportunity becomes a reload point.

This option will be written into Frequency of the Reload Opportunities bitfield (LDFQ) in Configuration register (CONF).

<note>

For safety, this option is activated only when PwmGeneral/PwmDutycycleUpdatedEndperiod is checked.

</note>

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.51 Parameter PwmFtmInvertControl

Select when the Inverting Control register is synchronized.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_disabled
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_↔ Sync_Trigger']

4.52 Parameter PwmFtmSwOutControl

Select when the Software Output Control register is synchronized.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_disabled
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_↔ Sync_Trigger']

4.53 Parameter PwmFtmOutputMask

Select when the Output Mask register is synchronized.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_disabled
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_↔ Sync_Trigger']

4.54 Parameter PwmFtmCounterInitial

Select when the Counter Initial, max counter, halfCycle and Counter Value register is synchronized.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_on_FTM_Sync_Trigger
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_↔ Sync_Trigger']

4.55 Parameter PwmFtmCounterSync

Select when the Ftm Counter register is synchronized.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_on_FTM_Sync_Trigger
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_↔ Sync_Trigger']

4.56 Container FtmGlobalFaultCfg

Global Fault configuration.

Autosar Requirements: Not specified.

Included subcontainers:

- [FtmFaultSettings](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.57 Parameter FtmFaultFunctionality

Selects the fault control mode

> `FAULT_DISABLED` -> Fault control disabled for all channels.

> `FAULT_EVEN_CHANNELS_MANUAL_CLEAR` -> Fault control enabled for even channels (0, 2, 4..) with manual fault clearing.

> `FAULT_ALL_CHANNELS_MANUAL_CLEAR` -> Fault control enabled for all channels with manual fault clearing.

> `FAULT_ALL_CHANNELS_AUTO_CLEAR` -> Fault control enabled for all channels with automatic fault clearing.

Note: For any automatic fault clear modes, a Fault Notification function should be defined. In automatic modes, the hardware shall

be configured into manual fault clearing and automatic fault clearing shall be implemented by software.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	<code>FAULT_DISABLED</code>
literals	<code>['FAULT_DISABLED', 'FAULT_EVEN_CHANNELS_MANUAL_CLEAR', 'FAULT_ALL_CHANNELS_MANUAL_CLEAR', 'FAULT_ALL_CHANNELS_AUTO_CLEAR']</code>

4.58 Parameter FtmFaultFilterValue

Selects the filter value for the fault inputs.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.59 Parameter FtmFaultOutputState

Specifies the fault state for the PWM channel output during fault conditions

SAFE_VALUE: Outputs will be placed into safe values when fault events in ongoing (defined by polarity)

TRI_STATE: Outputs will be tri-stated when fault event is ongoing

Autosar Requirements: Not specified.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	SAFE_VALUE
literals	['SAFE_VALUE', 'TRI_STATE']

4.60 Parameter FtmFaultIrqUsed

Enable/Disable the generation of an the fault notification.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.61 Container FtmFaultSettings

Configuration of an individual Ftm fault input.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	4
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.62 Parameter FtmFaultId

Select the Ftm Fault Id available in module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FLT_0
literals	['FLT_0', 'FLT_1', 'FLT_2', 'FLT_3']

4.63 Parameter FtmFaultPolarity

Defines the polarity of the fault input.

Autosar Requirements: not specified.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	POLARITY_HIGH
literals	['POLARITY_HIGH', 'POLARITY_LOW']

4.64 Parameter FtmEnableFaultFilter

Enables/Disables the filter for the fault input.

Autosar Requirements: Not specified.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.65 Parameter FtmFaultNotification

User callback function.

Note: Please use NULL or NULL_PTR w/o any quotes. If the used string is different from NULL or NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.66 Container PwmFtmCh

Configuration of an individual Ftm hardware channel.

Included subcontainers:

- [PwmFtmChInterruptCallback](#)
- [PwmFtmPairedCh](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.67 Parameter PwmFtmChId

Select the Ftm hardware channel available in module. This value will be assigned to the symbolic name derived of the PwmChannel container short name.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CH_0
literals	['CH_0', 'CH_1', 'CH_2', 'CH_3', 'CH_4', 'CH_5']

4.68 Parameter PwmFtmChPulseType

The type of the generated PWM signal.

For Edge/Center aligned PWM:

High-true pulse - clear Output on match

Low-true pulse - set Output on match

For combined modes:

High-true pulse - set on channel (n) match, and clear on channel (n+1) match

Low-true pulse - clear on channel (n) match, and set on channel (n+1) match

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	High_true_pulse
literals	['High_true_pulse', 'Low_true_pulse']

4.69 Parameter PwmFtmChInitOutput

The initial state of the channel output is configured in PWM signal.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	LOW
literals	['LOW', 'HIGH']

4.70 Parameter PwmFtmChPolarity

Select operation mode of this Ftm channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Channel_active_HIGH
literals	['Channel_active_LOW', 'Channel_active_HIGH']

4.71 Parameter PwmFtmChDutyCycle

Define the duty cycle value (in tick) for this channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65534
min	0

4.72 Parameter PwmFtmChDutyCycleDither

Define the duty cycle dither value (in tick) for this channel.

Valid value: [0, 1023]

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.73 Parameter PwmFtmChInterrupt

Enable/Disable the generation of an initialization trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.74 Parameter PwmFtmChPwmOutputEnable

Enable/Disable the output of the PWM signal on the channel pin.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.75 Parameter PwmFtmChSwCtrlEnable

Enable/Disable the software control for this channels output.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.76 Parameter PwmFtmChExternTriggerEnable

Enable/Disable the generation of an external trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.77 Parameter PwmFtmChSwCtrlValue

The software output control forces 0 or 1 to the channel output.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.78 Parameter PwmFtmChMatchLoadPointEnable

Enable/Disable the channel match as an reload opportunity.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.79 Parameter PwmFtmPairChEnable

Enable the paired channel feature.

The channels n and $n+1$ will be used as a channel pair.

Note: Cannot be used for odd channels.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.80 Container PwmFtmChInterruptCallback

Configure Notification function and parameter for interrupt handler callback.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.81 Parameter PwmFtmChInterruptFunctionCallback

User callback function.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.82 Parameter PwmFtmChInterruptParameterCallback

User callback parameter.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured parameter name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.83 Container PwmFtmPairedCh

Configuration of a paired Ftm hardware channel.

This will configure the channel n+1.

Included subcontainers:

- None
-

Property	Value
----------	-------

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.84 Parameter PwmFtmPairedChDeadtimeEnable

Enable the deadtime delay for Ftm channel.

The deadtime delay insertion ensures that no two complementary signals (channels n and n+1) drive the active state at the same time.

This option is activated only when Ftm channel is using operation mode that has COMPLEMENTARY (COMBINED_COMPLEMENTARY, PHASE_SHIFTED_COMPLEMENTARY).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.85 Parameter PwmFtmPairedChSync

Enable the synchronization of the channel match value registers for this pair.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.86 Parameter PwmFtmPairedChExtTrig

Enable the generation of an external trigger for channel n+1 of this pair.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.87 Parameter PwmFtmPairedChComplementary

Enable the complementary mode for Ftm channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.88 Parameter PwmFtmDutyDitheringPaired

The PWM edge dithering in Combine/Modified Combine Mode can be done in the channel (n) match edge or in the channel (n+1) match edge.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.89 Parameter PwmFtmPairedChCombineMode

Select combine mode for the Ftm channel pair.

NOTE:Combine and Modified Combine mode are not supported in Center-Aligned mode

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Combine_modes_disabled
literals	['Combine_modes_disabled', 'Combined_Mode', 'Modified_Combine_Mode']

4.90 Parameter PwmFtmPairedChComplementaryMode

Select complementary mode of the Ftm channel n+1.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Invert_Output
literals	['Invert_Output', 'Duplicate_Output']

4.91 Parameter PwmFtmPairedChPhaseShift

Define the offset value (in tick) from 1 that the leading edge of pulse signal will start.

Valid value: [0, 65533]

NOTE:Phase Shift is not supported in Center-Aligned mode

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65533
min	0

4.92 Container PwmGeneral

Container used for storing the general configuration of Pwm.

Included subcontainers:

- [PwmPowerStateConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.93 Parameter PwmMulticoreEnabled

Switch to enable/disable multicore feature.

User can choose ENABLE multicore feature by checking this option, this will force to configure at least 1 ECUC partition in

PwmChannelEcucPartitionRef, and each Pwm channel in PwmChannel to configure at least 1 ECUC partition reference

in PwmChannelEcucPartitionRef container to fulfill generating code condition; OR uncheck this option to DISABLE

multicore feature, performing this action will force user to remove all ECUC partition reference in every Pwm channels contained

in PwmChannel and in PwmChannelEcucPartitionRef.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.94 Parameter PwmDevErrorDetect

Switch to enable/disable the development error detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.95 Parameter PwmDutycycleUpdatedEndperiod

Switch to enable the update of the duty cycle parameter at the end of the current period.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.96 Parameter PwmPeriodUpdatedEndperiod

Switch to enable the update of the period parameter at the end of the current period.

<note>In current implementation, this option is locked and always enable by default.</note>

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.97 Parameter PwmNotificationSupported

Switch to indicate that the notifications are supported.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.98 Parameter PwmEnableUserModeSupport

If enabled, the Pwm module will adapt to run from User Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.99 Parameter PwmLowPowerStatesSupport

If enabled, hardware offers low power mode and adds all power state management related APIs

(Pwm_SetPowerState, Pwm_GetCurrentPowerState, Pwm_GetTargetPowerState,

Pwm_PreparePowerState, Pwm_Main_PowerTransitionManager),

indicating if the hardware offers low power state management.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.100 Parameter PwmPowerStateAsynchTransitionMode

Enable the support of the Pwm driver to the asynchronous power state transition.

<note> This feature is not supported and is rejected, as all hardware modules do not support asynchronous power state transitions.</note>

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.101 Parameter PwmEnableDualClockMode

Switch to enable/disable dual clock mode feature, which will add/remove the service Pwm_SetClockMode() from the code.

This feature is used when the prescaler value needs to be changed to maintain same period at different frequency.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.102 Parameter PwmEnableExternalTrigger

Enable external trigger generation.

This option will activate the use of PwmEnableTrigger and PwmDisableTrigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.103 Parameter PwmEnablePhaseShift

Enable phase-shift feature.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.104 Parameter PwmEnableMaskOutputs

Enable mask output feature, which will add/remove the services Pwm_MaskOutputs and Pwm_UnMaskOutputs from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.105 Parameter PwmFaultSupported

This enables the fault functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.106 Parameter PwmReloadNotificationSupported

This enables the reload point interrupt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.107 Parameter PwmMultiChannelSync

Enable the update synchronous feature for several channels.

This option will activate the use of PwmSetDutyCycle_NoUpdate, PwmSetPeriodAndDuty_NoUpdate and PwmSetDutyPhaseShift.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.108 Parameter PwmIndex

Specify the InstanceId of this module instance. If only one instance is present it shall have the ID 0.

<note>In current implementation, this feature is not used.</note>

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.109 Reference PwmEcucPartitionRef

Maps the Pwm driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Depending on the addressed timer resource the interfaces operate as follows.

When users choose ENABLE multicore feature by checking PwmMulticoreEnabled option, this will force to configure at least 1 ECUC partition in this list that is referenced from ECUC module; OR when DISABLE multicore feature, user have to remove all ECUC partitions in this list.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.110 Reference PwmKernelEcucPartitionRef

Maps the Pwm kernel to zero or one ECUC partitions to assign the driver kernel to a certain core.

The ECUC partition referenced is a subset of the ECUC partitions where the Pwm driver is mapped to.

<note>This feature is not supported and is rejected.</note>

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.111 Container PwmPowerStateConfig

Each instance of this parameter defines a power state and the callback to be called when this power state is reached.

<note>This feature is not supported and is rejected, as all hardware modules do not support asynchronous power state transitions.</note>

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.112 Parameter PwmPowerState

Each instance of this parameter describes a different power state supported by the Pwm hardware.

It should be defined by the hardware supplier and used by the Pwm Driver to reference specific HW configurations which set the Pwm HW module in the referenced power state.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	9223372036854775807
min	0

4.113 Parameter PwmPowerStateReadyCbkRef

Each instance of this parameter contains a reference to a power mode callback defined in a CDD or IoHwAbs component.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	NULL_PTR

4.114 Container PwmConfigurationOfOptApiServices

Container used for storing the configuration of all optional API's.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.115 Parameter PwmDeInitApi

Add/remove the service Pwm_DeInit() from the code.

This option will toggle PWM_DE_INIT_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.116 Parameter PwmGetOutputState

Add/remove the service Pwm_GetOutputState() from the code.

This option will toggle PWM_GET_OUTPUT_STATE_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.117 Parameter PwmSetDutyCycle

Add/remove the service Pwm_SetDutyCycle() from the code.

This option will toggle PWM_SET_DUTY_CYCLE_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.118 Parameter PwmSetOutputToIdle

Add/remove the service Pwm_SetOutputToIdle() from the code.

This option will toggle PWM_SET_OUTPUT_TO_IDLE_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.119 Parameter PwmSetPeriodAndDuty

Add/remove the service Pwm_SetPeriodAndDuty() from the code.

This option will toggle PWM_SET_PERIOD_AND_DUTY_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.120 Parameter PwmVersionInfoApi

Add/remove the service Pwm_GetVersionInfo() from the code.

This option will toggle PWM_VERSION_INFO_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.121 Parameter PwmGetChannelStateApi

Add/remove the service Pwm_GetChannelState() from the code.

This option will toggle PWM_GET_CHANNEL_STATE_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.122 Parameter PwmSetDutyCycle_NoUpdate

Add/remove the service Pwm_SetDutyCycle_NoUpdate() from the code.

This option will toggle PWM_SET_DUTY_CYCLE_NO_UPDATE_API define macro in Pwm_Cfg.h file.

<note>This option is activated only when PwmGeneral/PwmMultiChannelSync is enabled.</note>

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.123 Parameter PwmSetPeriodAndDuty_NoUpdate

Add/remove the service Pwm_SetPeriodAndDuty_NoUpdate() from the code.

This option will toggle PWM_SET_PERIOD_AND_DUTY_NO_UPDATE_API define macro in Pwm_Cfg.h file.

<note>This option is activated only when PwmGeneral/PwmMultiChannelSync is enabled.</note>

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.124 Parameter PwmSetDutyPhaseShift

Add/remove the service Pwm_SetDutyPhaseShift() from the code.

This option will toggle PWM_SET_DUTY_PHASE_SHIFT_API define macro in Pwm_Cfg.h file.

<note>This option is activated only when PwmGeneral/PwmMultiChannelSync is enabled.</note>

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.125 Parameter PwmSetChannelDeadTime

Add/remove the service Pwm_SetChannelDeadTime() from the code.

This option will toggle PWM_SETCHANNELDEADTIME_API define macro.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.126 Parameter PwmForceOutputToZeroApi

Add/remove the service Pwm_ForceOutputToZero() from the code.

This option will toggle PWM_FORCE_OUTPUT_TO_ZERO_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.127 Parameter PwmEnableTrigger

Add/remove the service Pwm_EnableTrigger() from the code.

This option will toggle PWM_ENABLE_TRIGGER_API define macro in Pwm_Cfg.h file.

<note>This option is activated only when PwmGeneral/PwmEnableExternalTrigger is enabled.</note>

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.128 Parameter PwmDisableTrigger

Add/remove the service Pwm_DisableTrigger() from the code.

This option will toggle PWM_DISABLE_TRIGGER_API define macro in Pwm_Cfg.h file.

<note>This option is activated only when PwmGeneral/PwmEnableExternalTrigger is enabled.</note>

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.129 Parameter PwmResetCounter

Add/remove the service Pwm_ResetCounterEnable() and Pwm_ResetCounterDisable() from the code.

This option will toggle PWM_RESET_COUNTER_API define macro in Pwm_Cfg.h file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.130 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.131 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.132 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.133 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.134 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	121
max	121
min	121

4.135 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.136 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.137 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2

Property	Value
max	2
min	2

4.138 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

`<ModuleName>_<VendorId>_<VendorApiInfix>`.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.139 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Ftm Pwm IPL	95
Pwm Driver	122

Chapter 6

Module Documentation

6.1 Ftm Pwm IPL

6.1.1 Detailed Description

Data Structures

- struct [Ftm_Pwm_Ip_CallbackCfgType](#)
Structure for notification. [More...](#)
- struct [Ftm_Pwm_Ip_SyncCfgType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_InstanceCfgType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_PairCfgType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_ChannelConfigType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_UserCfgType](#)
Ftm configuration parameters structure. [More...](#)

Macros

- `#define` [FTM_PWM_IP_MAX_DUTY_CYCLE](#)
Maximum value for PWM duty cycle.
- `#define` [FTM_PWM_IP_NO_EDGE](#)
FlexTimer edge interrupt.

Types Reference

- typedef void(* [Ftm_Pwm_Ip_CallbackType](#)) (uint8 CbParam)
Notification function callback type.

Enum Reference

- enum [Ftm_Pwm_Ip_StatusType](#)
Status return codes.
- enum [Ftm_Pwm_Ip_ChannelModeType](#)
FlexTimer operation mode.
- enum [Ftm_Pwm_Ip_ChannelStateType](#)
FlexTimer channel state.
- enum [Ftm_Pwm_Ip_PrescalerType](#)
Prescaler type.
- enum [Ftm_Pwm_Ip_PowerStateType](#)
Power state type.
- enum [Ftm_Pwm_Ip_NotifType](#)
Notification type.
- enum [Ftm_Pwm_Ip_CounterModeType](#)
FlexTimer operation mode.
- enum [Ftm_Pwm_Ip_ClkSourceType](#)
FlexTimer clock source selection.
- enum [Ftm_Pwm_Ip_PolarityType](#)
The polarity of the channel output is configured in PWM signal.
- enum [Ftm_Pwm_Ip_OutputStateType](#)
The initial state of the channel output is configured in PWM signal.
- enum [Ftm_Pwm_Ip_ClkPrescalerType](#)
FlexTimer pre-scaler factor selection for the clock source.
- enum [Ftm_Pwm_Ip_SyncType](#)
FTM sync source.
- enum [Ftm_Pwm_Ip_SyncModeType](#)
FTM sync mode.
- enum [Ftm_Pwm_Ip_DeadtimePrescalerType](#)
FlexTimer pre-scaler factor for the dead-time insertion.
- enum [Ftm_Pwm_Ip_DebugModeType](#)
Options for the FlexTimer behavior in BDM Mode.
- enum [Ftm_Pwm_Ip_InitTriggModeType](#)
FlexTimer PWM initialization trigger generation mode.
- enum [Ftm_Pwm_Ip_PairChPolarityType](#)
FlexTimer PWM channel (n+1) polarity for combine mode.
- enum [Ftm_Pwm_Ip_PairDitherEdgeType](#)
FlexTimer PWM Pair Edge Dithering for combine/mcombine modes.

Function Reference

- void [Ftm_Pwm_Ip_Init](#) (uint8 Instance, const [Ftm_Pwm_Ip_UserCfgType](#) *UserCfg)
Initializes the FTM driver. The clock sources depend on the specified hardware. So that needs to check the external clock or the fixed frequency clock is available on device or not.
- void [Ftm_Pwm_Ip_DeInit](#) (uint8 Instance)
Shuts down the FTM driver.

- [Ftm_Pwm_Ip_StatusType Ftm_Pwm_Ip_UpdatePwmChannel](#) (uint8 Instance, uint8 Channel, uint16 FirstEdge, uint16 SecondEdge, boolean SoftwareTrigger)
This function updates the waveform output in PWM mode (duty cycle and phase).
- void [Ftm_Pwm_Ip_FastUpdatePwmDuty](#) (uint8 Instance, uint8 NumberOfChannels, const uint8 *Channels, const uint16 *Duty, boolean SoftwareTrigger)
This function will update the duty cycle of PWM output for multiple channels.
- [Ftm_Pwm_Ip_StatusType Ftm_Pwm_Ip_UpdatePwmPeriod](#) (uint8 Instance, uint32 NewValue, boolean SoftwareTrigger)
This function will update the new period in the frequency or in the counter value into mode register which modify the period of PWM signal on the Channel output.
- void [Ftm_Pwm_Ip_SwOutputControl](#) (uint8 Instance, uint8 Channel, [Ftm_Pwm_Ip_OutputStateType](#) OutputState, boolean ActiveState)
This function performs the update of the SWOCTRL register on request to set/clear output state of channel in idle state or output-forced.
- void [Ftm_Pwm_Ip_SyncUpdate](#) (uint8 Instance)
Ftm_Pwm_Ip_SyncUpdate.
- void [Ftm_Pwm_Ip_DisableNotification](#) (uint8 Instance, [Ftm_Pwm_Ip_NotifType](#) NotifType)
This function disables the user notifications.
- void [Ftm_Pwm_Ip_EnableNotification](#) (uint8 Instance, [Ftm_Pwm_Ip_NotifType](#) NotifType)
This function enables the user notifications.
- void [Ftm_Pwm_Ip_SetClockMode](#) (uint8 Instance, [Ftm_Pwm_Ip_PrescalerType](#) Prescaler)
This function sets the value of the prescaler on all FTM modules.
- void [Ftm_Pwm_Ip_ResetCounter](#) (uint8 Instance, boolean IsResetCnt)
This function enabel/disabled counter reset by [Pwm_SyncUpdate\(\)](#) call.
- void [Ftm_Pwm_Ip_MaskOutputChannels](#) (uint8 Instance, uint32 ChannelsMask, boolean SoftwareTrigger)
This function will mask the output of the channels and at match events will be ignored by the masked channels.
- void [Ftm_Pwm_Ip_UpdatePwmDutyCycleChannel](#) (uint8 Instance, uint8 Channel, uint16 DutyCycle, boolean SoftwareTrigger)
This function sets the duty cycle for the specified PWM channel.
- void [Ftm_Pwm_Ip_UpdatePwmPeriodAndDuty](#) (uint8 Instance, uint8 Channel, uint16 Period, uint16 Duty↵ Cycle, boolean SoftwareTrigger)
This function sets the duty cycle and period for the specified PWM channel.
- [Ftm_Pwm_Ip_OutputStateType Ftm_Pwm_Ip_GetOutputState](#) (uint8 Instance, uint8 Channel)
This function used to get the current output state of the PWM channel.
- void [Ftm_Pwm_Ip_SetPowerState](#) (uint8 Instance, [Ftm_Pwm_Ip_PowerStateType](#) PowerState)
Put PWM channel based on FTM to target power mode.
- uint16 [Ftm_Pwm_Ip_GetChannelState](#) (uint8 Instance, uint8 Channel)
This function read the duty cycle value from the duty cycle array and returns this value.
- void [Ftm_Pwm_Ip_SetPhaseShift](#) (uint8 Instance, uint8 Channel, uint16 Period, uint16 PhaseShift, boolean SoftwareTrigger)
This function set phase shift value and also force duty cycle to 50%.
- void [Ftm_Pwm_Ip_SetDutyPhaseShift](#) (uint8 Instance, uint8 Channel, uint16 DutyCycle, uint16 PhaseShift, boolean SyncUpdate)
This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)
- void [Ftm_Pwm_Ip_UnMaskOutputChannels](#) (uint8 Instance, uint32 ChannelsMask, boolean Software↵ Trigger)
This function will unmask the output of the channels and at match events will be ignored by the masked channels.
- void [Ftm_Pwm_Ip_EnableTrigger](#) (uint8 Instance, uint32 TriggerMask)

This function enables trigger generation for specific source.

- void [Ftm_Pwm_Ip_DisableTrigger](#) (uint8 Instance, uint32 TriggerMask)

This function disables trigger generation for specific source.

- void [Ftm_Pwm_Ip_SetChannelDeadTime](#) (uint8 Instance, uint8 Channel, uint16 DeadTimeVal)

This function is used to update the deadtime at runtime for Pwm channels.

6.1.2 Data Structure Documentation

6.1.2.1 struct Ftm_Pwm_Ip_CallbackCfgType

Structure for notification.

The structure used to notification

Definition at line 345 of file Ftm_Pwm_Ip_Types.h.

6.1.2.2 struct Ftm_Pwm_Ip_SyncCfgType

Ftm configuration parameters structure.

FlexTimer Sync and Load parameters

Definition at line 374 of file Ftm_Pwm_Ip_Types.h.

Data Fields

Type	Name	Description
Ftm_Pwm_Ip_SyncModeType	SyncMode	Configure FTM Sync to use Software or Hardware trigger modes
boolean	HwSync0	Enable/disable hardware 0 sync
boolean	HwSync1	Enable/disable hardware 1 sync
boolean	HwSync2	Enable/disable hardware 2 sync
boolean	HwTriggerAutoClear	Enable/disable auto clear of triggers in hardware trigger mode
boolean	MaxLoadPoint	Enable/disable maximum loading point
boolean	MinLoadPoint	Enable/disable minimum loading point
boolean	HalfCycleLoadPoint	Enable/disable half cycle loading point
uint16	HalfCycleLoadPointValue	The match value of FTM half cycle reload feature
uint8	LoadPointFreq	Value of how many reload points must happen before reloading.
Ftm_Pwm_Ip_SyncType	InverterSync	Configures INVCTRL sync
Ftm_Pwm_Ip_SyncType	OutRegSync	Configures SWOCTRL sync
Ftm_Pwm_Ip_SyncType	OutMaskSync	Configures OUTMASK sync
Ftm_Pwm_Ip_SyncType	InitCounterSync	Configures CNTIN sync
Ftm_Pwm_Ip_SyncType	CounterSync	Configures FTM Counter sync

6.1.2.3 struct Ftm_Pwm_Ip_InstanceCfgType

Ftm configuration parameters structure.

Ftm IP specific instance configuration structure type

Definition at line 402 of file Ftm_Pwm_Ip_Types.h.

Data Fields

Type	Name	Description
Ftm_Pwm_Ip_ClkSourceType	ClkSrc	Clock source for Ftm instances
Ftm_Pwm_Ip_ClkPrescalerType	ClkPs	Pre-scaler for Ftm instances
Ftm_Pwm_Ip_ClkPrescalerType	AlternateClkPs	Alternative pre-scaler for Ftm instances
Ftm_Pwm_Ip_CounterModeType	CntMode	Mode of the counter
boolean	OverflowIrqEn	Selected the overflow interrupt
Ftm_Pwm_Ip_CallbackCfgType	OverflowCb	overflow callback
boolean	ReloadIrqEn	Selected the reload interrupt
Ftm_Pwm_Ip_ReloadCallbackCfgType	ReloadCb	Reload callback
Ftm_Pwm_Ip_DebugModeType	DebugMode	Debug Mode
boolean	WriteProtection	Selected write protection
boolean	InitTriggerEn	Selected trigger
Ftm_Pwm_Ip_InitTriggModeType	InitTrigMode	Chose generation of initialization trigger mode
uint16	PwmPeriod	Period of Ftm instances
uint8	PwmPeriodDither	Period Dither
uint16	DeadTimeValue	Deadtime Value
Ftm_Pwm_Ip_DeadtimePrescalerType	DeadTimePs	Deadtime pre-scaler Value
const Ftm_Pwm_Ip_SyncCfgType *	SyncCfg	Pointer to the configured channels for the FTM sync

6.1.2.4 struct Ftm_Pwm_Ip_PairCfgType

Ftm configuration parameters structure.

FlexTimer Channel Pair configuration parameters

Definition at line 438 of file Ftm_Pwm_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	PairId	FTM pair id channel.
boolean	DeadtimeEn	Selected dead-time for pair channel.
boolean	ComplementaryModeEn	Selected Complementary mode for pair channel.

Data Fields

Type	Name	Description
Ftm_Pwm_Ip_PairChPolarityType	PairChPolarity	Polarity of pair channel.
boolean	PairExtTrigEn	Selected external trigger of pair channel.
boolean	PairSyncEn	Selected sync of pair channel.
uint16	PhaseShiftValue	the phase Shift Value
Ftm_Pwm_Ip_PairDitherEdgeType	DitherEdge	

6.1.2.5 struct Ftm_Pwm_Ip_ChannelConfigType

Ftm configuration parameters structure.

FlexTimer Channel configuration parameters

Definition at line 464 of file Ftm_Pwm_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	ChannelId	FTM channel ID.
Ftm_Pwm_Ip_ChannelModeType	ChannelMode	Mode of Ftm channel.
boolean	ChIrqEn	Selected for interrupt.
Ftm_Pwm_Ip_CallbackCfgType	ChannelCb	Callback for the ftm channels.
boolean	ChOutputEn	Enable/Disable the output of the PWM signal on the channel pin.
boolean	SwControlEn	Enable/Disable the software control for this channels output.
boolean	SwControlVal	Channel Software Output Control Value.
boolean	ExtTrigEn	Selected for external trigger.
boolean	ChMatchLoadEn	Selected for external trigger.
uint16	DutyCycle	Duty cycle value.
uint8	DutyCycleDither	Duty cycle value in Dither.
Ftm_Pwm_Ip_OutputStateType	InitOut	Output state of Ftm channel.
Ftm_Pwm_Ip_PolarityType	Polarity	Polarity of Ftm channel.
const Ftm_Pwm_Ip_PairCfgType *	PairCfg	The configuration of pair channel.

6.1.2.6 struct Ftm_Pwm_Ip_UserCfgType

Ftm configuration parameters structure.

FlexTimer User Configuration structure

Definition at line 529 of file Ftm_Pwm_Ip_Types.h.

Data Fields

Type	Name	Description
const Ftm_Pwm_Ip_InstanceCfgType *	InstanceCfg	Pointer to the configured channels for the FTM instance.
const Ftm_Pwm_Ip_ChannelConfigType *const *	ConfiguredChArray	Pointer to the configured channels array for the FTM channel.
uint8	NoOfConfiguredCh	Number of the configured channels for the FTM channel.

6.1.3 Macro Definition Documentation

6.1.3.1 FTM_PWM_IP_MAX_DUTY_CYCLE

```
#define FTM_PWM_IP_MAX_DUTY_CYCLE
```

Maximum value for PWM duty cycle.

Definition at line 121 of file [Ftm_Pwm_Ip.h](#).

6.1.3.2 FTM_PWM_IP_NO_EDGE

```
#define FTM_PWM_IP_NO_EDGE
```

FlexTimer edge interrupt.

Definition at line 106 of file [Ftm_Pwm_Ip_Types.h](#).

6.1.4 Types Reference

6.1.4.1 Ftm_Pwm_Ip_CallbackType

```
typedef void(* Ftm_Pwm_Ip_CallbackType) (uint8 CbParam)
```

Notification function callback type.

Definition at line 338 of file [Ftm_Pwm_Ip_Types.h](#).

6.1.5 Enum Reference

6.1.5.1 Ftm_Pwm_Ip_StatusType

```
enum Ftm\_Pwm\_Ip\_StatusType
```

Status return codes.

Enumerator

FTM_PWM_IP_STATUS_SUCCESS	Generic operation success status
FTM_PWM_IP_STATUS_ERROR	Generic operation failure status
FTM_PWM_IP_STATUS_BUSY	Generic operation busy status
FTM_PWM_IP_STATUS_TIMEOUT	Generic operation timeout status
FTM_PWM_IP_STATUS_UNSUPPORTED	Generic operation unsupported status

Definition at line 113 of file Ftm_Pwm_Ip_Types.h.

6.1.5.2 Ftm_Pwm_Ip_ChannelModeType

enum `Ftm_Pwm_Ip_ChannelModeType`

FlexTimer operation mode.

Enumerator

FTM_PWM_IP_MODE_EDGE_ALIGNED_HIGH	Edge aligned PWM High-true pulses (clear on match)
FTM_PWM_IP_MODE_EDGE_ALIGNED_LOW	Edge aligned PWM Low-true pulses (set on match)
FTM_PWM_IP_MODE_CENTER_ALIGNED_↔ HIGH	Center aligned PWM High-true pulses (clear on match)
FTM_PWM_IP_MODE_CENTER_ALIGNED_↔ LOW	Center aligned PWM Low-true pulses (set on match)
FTM_PWM_IP_MODE_COMBINE_HIGH	Combined channels PWM High-true pulses (set on channel (n) match, and clear on channel (n+1) match)
FTM_PWM_IP_MODE_COMBINE_LOW	Combined channels PWM Low-true pulses (clear on channel (n) match, and set on channel (n+1) match)
FTM_PWM_IP_MODE_MODIFIED_COMBIN↔ E_HIGH	Modified combined channel PWM High-true pulses (set on channel (n) match, and clear on channel (n+1) match)
FTM_PWM_IP_MODE_MODIFIED_COMBIN↔ E_LOW	Modified combined channel PWM Low-true pulses (clear on channel (n) match, and set on channel (n+1) match)
FTM_PWM_IP_MODE_NODEFINED	Non defined Mode

Definition at line 124 of file Ftm_Pwm_Ip_Types.h.

6.1.5.3 Ftm_Pwm_Ip_ChannelStateType

enum `Ftm_Pwm_Ip_ChannelStateType`

FlexTimer channel state.

Enumerator

FTM_PWM_IP_CHANNEL_UNINIT	The state's channel before initialization
FTM_PWM_IP_CHANNEL_RUNNING	The state's channel is running
FTM_PWM_IP_CHANNEL_IDLE	The state's channel is idle
FTM_PWM_IP_CHANNEL_OUTPUT_FORCED	The state's channel in force output channel mode

Definition at line 141 of file Ftm_Pwm_Ip_Types.h.

6.1.5.4 Ftm_Pwm_Ip_PrescalerType

```
enum Ftm_Pwm_Ip_PrescalerType
```

Prescaler type.

This enumeration specifies the possible types of prescalers used to configure base-clock timers

Enumerator

FTM_PWM_IP_PRIMARY_PRESCALER	Selected value is the default/primary prescaler.
FTM_PWM_IP_ALTERNATIVE_PRESCALER	Selected value is the alternative configured prescaler.

Definition at line 153 of file Ftm_Pwm_Ip_Types.h.

6.1.5.5 Ftm_Pwm_Ip_PowerStateType

```
enum Ftm_Pwm_Ip_PowerStateType
```

Power state type.

Power state currently active or set as target power state.

Enumerator

FTM_PWM_IP_FULL_POWER	PWM full power mode.
FTM_PWM_IP_LOW_POWER	PWM low power mode.
FTM_PWM_IP_NODEFINE_POWER	PWM no define power mode.

Definition at line 166 of file Ftm_Pwm_Ip_Types.h.

6.1.5.6 Ftm_Pwm_Ip_NotifType

enum `Ftm_Pwm_Ip_NotifType`

Notification type.

This enumeration defines the type of notification

Enumerator

FTM_PWM_IP_CHANNEL_0_NOTIFICATION	A notification will be generated when an event occurs on the channel 0.
FTM_PWM_IP_CHANNEL_1_NOTIFICATION	A notification will be generated when an event occurs on the channel 1.
FTM_PWM_IP_CHANNEL_2_NOTIFICATION	A notification will be generated when an event occurs on the channel 2.
FTM_PWM_IP_CHANNEL_3_NOTIFICATION	A notification will be generated when an event occurs on the channel 3.
FTM_PWM_IP_CHANNEL_4_NOTIFICATION	A notification will be generated when an event occurs on the channel 4.
FTM_PWM_IP_CHANNEL_5_NOTIFICATION	A notification will be generated when an event occurs on the channel 5.
FTM_PWM_IP_CHANNEL_6_NOTIFICATION	A notification will be generated when an event occurs on the channel 6.
FTM_PWM_IP_CHANNEL_7_NOTIFICATION	A notification will be generated when an event occurs on the channel 7.
FTM_PWM_IP_OVERFLOW_NOTIFICATION	A notification will be generated when the FTM counter passes the value in the MOD register.
FTM_PWM_IP_RELOAD_POINT_NOTIFICATION	A notification will be generated when a selected reload point happened.
FTM_PWM_IP_FAULT_NOTIFICATION	A notification will be generated when an event for the FTM peripheral fault channel has occurred.

Definition at line 181 of file `Ftm_Pwm_Ip_Types.h`.

6.1.5.7 Ftm_Pwm_Ip_CounterModeType

enum `Ftm_Pwm_Ip_CounterModeType`

FlexTimer operation mode.

Enumerator

FTM_PWM_IP_EDGE_ALIGNED	The FTM Counter is in UP Counting mode
FTM_PWM_IP_CENTER_ALIGNED	The FTM Counter is in UP-DOWN Counting mode

Definition at line 208 of file Ftm_Pwm_Ip_Types.h.

6.1.5.8 Ftm_Pwm_Ip_ClkSourceType

```
enum Ftm_Pwm_Ip_ClkSourceType
```

FlexTimer clock source selection.

Enumerator

FTM_PWM_IP_CLOCK_SOURCE_NONE	None use clock for FTM
FTM_PWM_IP_CLOCK_SOURCE_SYSTEMCLK	System clock
FTM_PWM_IP_CLOCK_SOURCE_FIXEDCLK	Fixed clock
FTM_PWM_IP_CLOCK_SOURCE_EXTERNALCLK	External clock

Definition at line 215 of file Ftm_Pwm_Ip_Types.h.

6.1.5.9 Ftm_Pwm_Ip_PolarityType

```
enum Ftm_Pwm_Ip_PolarityType
```

The polarity of the channel output is configured in PWM signal.

Enumerator

FTM_PWM_IP_POLARITY_LOW	The channel polarity is active LOW
FTM_PWM_IP_POLARITY_HIGH	The channel polarity is active HIGH

Definition at line 228 of file Ftm_Pwm_Ip_Types.h.

6.1.5.10 Ftm_Pwm_Ip_OutputStateType

```
enum Ftm_Pwm_Ip_OutputStateType
```

The initial state of the channel output is configured in PWM signal.

Enumerator

FTM_PWM_IP_OUTPUT_STATE_LOW	The channel output is LOW
FTM_PWM_IP_OUTPUT_STATE_HIGH	The channel output is HIGH

Definition at line 235 of file Ftm_Pwm_Ip_Types.h.

6.1.5.11 Ftm_Pwm_Ip_ClkPrescalerType

```
enum Ftm_Pwm_Ip_ClkPrescalerType
```

FlexTimer pre-scaler factor selection for the clock source.

Enumerator

FTM_PWM_IP_CLOCK_DIV_1	Divide by 1
FTM_PWM_IP_CLOCK_DIV_2	Divide by 2
FTM_PWM_IP_CLOCK_DIV_4	Divide by 4
FTM_PWM_IP_CLOCK_DIV_8	Divide by 8
FTM_PWM_IP_CLOCK_DIV_16	Divide by 16
FTM_PWM_IP_CLOCK_DIV_32	Divide by 32
FTM_PWM_IP_CLOCK_DIV_64	Divide by 64
FTM_PWM_IP_CLOCK_DIV_128	Divide by 128

Definition at line 242 of file Ftm_Pwm_Ip_Types.h.

6.1.5.12 Ftm_Pwm_Ip_SyncType

```
enum Ftm_Pwm_Ip_SyncType
```

FTM sync source.

Enumerator

FTM_PWM_IP_SYNC_DISABLED	synchronization for the FTM register is disabled
FTM_PWM_IP_SYNC_ON_CLK	synchronization for the FTM register is done on each FTM Clock rising edge
FTM_PWM_IP_SYNC_ON_TRIGGER	synchronization for the FTM register is done on each FTM Sync Trigger

Definition at line 255 of file Ftm_Pwm_Ip_Types.h.

6.1.5.13 Ftm_Pwm_Ip_SyncModeType

enum `Ftm_Pwm_Ip_SyncModeType`

FTM sync mode.

Enumerator

FTM_PWM_IP_SYNC_SWTRIGGER	synchronization trigger for FTM is generated by software
FTM_PWM_IP_SYNC_HWTRIGGER	synchronization trigger for FTM is generated by hardware

Definition at line 263 of file Ftm_Pwm_Ip_Types.h.

6.1.5.14 Ftm_Pwm_Ip_DeadtimePrescalerType

enum `Ftm_Pwm_Ip_DeadtimePrescalerType`

FlexTimer pre-scaler factor for the dead-time insertion.

Enumerator

FTM_PWM_IP_DEADTIME_DIV_1	Divide by 1
FTM_PWM_IP_DEADTIME_DIV_4	Divide by 4
FTM_PWM_IP_DEADTIME_DIV_16	Divide by 16

Definition at line 270 of file Ftm_Pwm_Ip_Types.h.

6.1.5.15 Ftm_Pwm_Ip_DebugModeType

enum `Ftm_Pwm_Ip_DebugModeType`

Options for the FlexTimer behavior in BDM Mode.

Enumerator

FTM_PWM_IP_BDM_MODE_00	FTM counter stopped, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers bypass the register buffers
FTM_PWM_IP_BDM_MODE_01	FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are forced to their safe value , writes to MOD,CNTIN and C(n)V registers bypass the register buffers
FTM_PWM_IP_BDM_MODE_10	FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are frozen when chip enters in BDM mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers
FTM_PWM_IP_BDM_MODE_11	FTM counter in functional mode, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers is in fully functional mode

Definition at line 278 of file Ftm_Pwm_Ip_Types.h.

6.1.5.16 Ftm_Pwm_Ip_InitTriggModeType

```
enum Ftm_Pwm_Ip_InitTriggModeType
```

FlexTimer PWM initialization trigger generation mode.

Enumerator

FTM_PWM_IP_INIT_TRIGG_RELOAD_POINT	Initialization trigger is generated when FTM counter reaches a reload point
FTM_PWM_IP_INIT_TRIGG_CNT_UPDATE	Initialization trigger is generated when FTM counter is updated

Definition at line 295 of file Ftm_Pwm_Ip_Types.h.

6.1.5.17 Ftm_Pwm_Ip_PairChPolarityType

```
enum Ftm_Pwm_Ip_PairChPolarityType
```

FlexTimer PWM channel (n+1) polarity for combine mode.

Enumerator

FTM_PWM_IP_MAIN_INVERTED	The channel (n+1) output is the inverse of the channel (n) output
FTM_PWM_IP_MAIN_DUPLICATED	The channel (n+1) output is the same as the channel (n) output

Definition at line 302 of file Ftm_Pwm_Ip_Types.h.

6.1.5.18 Ftm_Pwm_Ip_PairDitherEdgeType

```
enum Ftm_Pwm_Ip_PairDitherEdgeType
```

FlexTimer PWM Pair Edge Dithering for combine/mcombine modes.

Enumerator

FTM_PWM_IP_DITHER_LEADING_EDGE	The channel (n) dithering is used
FTM_PWM_IP_DITHER_TRAILING_EDGE	The channel (n+1) dithering is used

Definition at line 309 of file Ftm_Pwm_Ip_Types.h.

6.1.6 Function Reference

6.1.6.1 Ftm_Pwm_Ip_Init()

```
void Ftm_Pwm_Ip_Init (
    uint8 Instance,
    const Ftm_Pwm_Ip_UserCfgType * UserCfg )
```

Initializes the FTM driver. The clock sources depend on the specified hardware. So that needs to check the external clock or the fixed frequency clock is available on device or not.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>info</i>	The FTM user configuration structure, see #Ftm_Pwm_Ip_UserConfigType.
out	<i>state</i>	The FTM state structure of the driver.

Returns

void

6.1.6.2 Ftm_Pwm_Ip_DeInit()

```
void Ftm_Pwm_Ip_DeInit (
    uint8 Instance )
```

Shuts down the FTM driver.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
----	-----------------	-------------------------------------

Returns

void

6.1.6.3 Ftm_Pwm_Ip_UpdatePwmChannel()

```

Ftm_Pwm_Ip_StatusType Ftm_Pwm_Ip_UpdatePwmChannel (
    uint8 Instance,
    uint8 Channel,
    uint16 FirstEdge,
    uint16 SecondEdge,
    boolean SoftwareTrigger )

```

This function updates the waveform output in PWM mode (duty cycle and phase).

@Note: Regarding the type of updating PWM in the duty cycle, if the expected duty is 100% then the value that is to be written to hardware will be exceed value of period. It means that the FTM counter will not match the value of the CV register in this case.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The channel number. In combined mode, the code finds the channel.
in	<i>FirstEdge</i>	Duty cycle or first edge time for PWM mode. Can take value between 0 - FTM_PWM_IP_MAX_DUTY_CYCLE(0 = 0% from period and FTM_PWM_IP_MAX_DUTY_CYCLE = 100% from period) Or value in ticks for the first of the PWM mode in which can have value between 0 and ftmPeriod is stored in the state structure.
in	<i>SecondEdge</i>	Second edge time - only for combined mode. Can take value between 0 - FTM_PWM_IP_MAX_DUTY_CYCLE(0 = 0% from period and FTM_PWM_IP_MAX_DUTY_CYCLE = 100% from period). Or value in ticks for the second of the PWM mode in which can have value between 0 and ftmPeriod is stored in the state structure.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.
- FTM_PWM_STATUS_ERROR : Error occurred.

6.1.6.4 Ftm_Pwm_Ip_FastUpdatePwmDuty()

```
void Ftm_Pwm_Ip_FastUpdatePwmDuty (
    uint8 Instance,
    uint8 NumberOfChannels,
    const uint8 * Channels,
    const uint16 * Duty,
    boolean SoftwareTrigger )
```

This function will update the duty cycle of PWM output for multiple channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>NumberOfChannels</i>	The number of channels which should be updated.
in	<i>Channels</i>	The list of channels which should be updated.
in	<i>Duty</i>	The list of duty cycles for selected channels.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.
- FTM_PWM_STATUS_ERROR : Error occurred.

6.1.6.5 Ftm_Pwm_Ip_UpdatePwmPeriod()

```
Ftm_Pwm_Ip_StatusType Ftm_Pwm_Ip_UpdatePwmPeriod (
    uint8 Instance,
    uint32 NewValue,
    boolean SoftwareTrigger )
```

This function will update the new period in the frequency or in the counter value into mode register which modify the period of PWM signal on the Channel output.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>NewValue</i>	The frequency or the counter value which will select with modified value for PWM signal. If the type of update in the duty cycle, the newValue parameter must be value between 1U and maximum is the frequency of the FTM counter. If the type of update in ticks, the newValue parameter must be value between 1U and 0xFFFFU.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

operation status

- FTM_PWM_STATUS_SUCCESS : Completed successfully.
- FTM_PWM_STATUS_ERROR : Error occurred.

6.1.6.6 Ftm_Pwm_Ip_SwOutputControl()

```
void Ftm_Pwm_Ip_SwOutputControl (
    uint8 Instance,
    uint8 Channel,
    Ftm_Pwm_Ip_OutputStateType OutputState,
    boolean ActiveState )
```

This function performs the update of the SWOCTRL register on request to set/clear output state of channel in idle state or output-forced.

This function can be called from Pwm_Ftm_SetOutputToIdle, Pwm_Ftm_DeInit and Pwm_Ftm_ForceOutput↵ToZero functions

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index
in	<i>OutputState</i>	PWM output level
in	<i>ActiveState</i>	Activation state of the SW Output Control TRUE - Enable SW Output Control FALSE - Disable SW Output Control

Returns

void

6.1.6.7 Ftm_Pwm_Ip_SyncUpdate()

```
void Ftm_Pwm_Ip_SyncUpdate (
    uint8 Instance )
```

Ftm_Pwm_Ip_SyncUpdate.

This function will allow synchronized loading of the duty registers for all the channels of a given FTM module

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
----	-----------------	-------------------------------------

Returns

void

6.1.6.8 Ftm_Pwm_Ip_DisableNotification()

```
void Ftm_Pwm_Ip_DisableNotification (
    uint8 Instance,
    Ftm_Pwm_Ip_NotifType NotifType )
```

This function disables the user notifications.

This function disables the user notifications for the corresponding type of notification

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>NotifType</i>	The type of notification

Returns

void

6.1.6.9 Ftm_Pwm_Ip_EnableNotification()

```
void Ftm_Pwm_Ip_EnableNotification (
    uint8 Instance,
    Ftm_Pwm_Ip_NotifType NotifType )
```

This function enables the user notifications.

This function enables the user notifications for the corresponding type of notification

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>NotifType</i>	The type of notification

Returns

void

6.1.6.10 Ftm_Pwm_Ip_SetClockMode()

```
void Ftm_Pwm_Ip_SetClockMode (
    uint8 Instance,
    Ftm_Pwm_Ip_PrescalerType Prescaler )
```

This function sets the value of the prescaler on all FTM modules.

This function sets the prescaler depending on the value ePrescaler parameter.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Prescaler</i>	Prescaler values

Returns

void

6.1.6.11 Ftm_Pwm_Ip_ResetCounter()

```
void Ftm_Pwm_Ip_ResetCounter (
    uint8 Instance,
    boolean IsResetCnt )
```

This function enabel/disabled counter reset by [Pwm_SyncUpdate\(\)](#) call.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>IsResetCnt</i>	Reset counter or not

Returns

void

6.1.6.12 Ftm_Pwm_Ip_MaskOutputChannels()

```
void Ftm_Pwm_Ip_MaskOutputChannels (
    uint8 Instance,
    uint32 ChannelsMask,
    boolean SoftwareTrigger )
```

This function will mask the output of the channels and at match events will be ignored by the masked channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>ChannelsMask</i>	The mask which will select which channels will ignore match events.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.

6.1.6.13 Ftm_Pwm_Ip_UpdatePwmDutyCycleChannel()

```
void Ftm_Pwm_Ip_UpdatePwmDutyCycleChannel (
    uint8 Instance,
    uint8 Channel,
    uint16 DutyCycle,
    boolean SoftwareTrigger )
```

This function sets the duty cycle for the specified PWM channel.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The FTM peripheral channel number.
in	<i>DutyCycle</i>	PWM duty cycle value [0-0x8000]
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

void

6.1.6.14 Ftm_Pwm_Ip_UpdatePwmPeriodAndDuty()

```
void Ftm_Pwm_Ip_UpdatePwmPeriodAndDuty (
    uint8 Instance,
    uint8 Channel,
    uint16 Period,
    uint16 DutyCycle,
    boolean SoftwareTrigger )
```

This function sets the duty cycle and period for the specified PWM channel.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The FTM peripheral channel number.
in	<i>Period</i>	The period will be updated
in	<i>DutyCycle</i>	PWM duty cycle value [0-0x8000]
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

void

6.1.6.15 Ftm_Pwm_Ip_GetOutputState()

```
Ftm_Pwm_Ip_OutputStateType Ftm_Pwm_Ip_GetOutputState (
    uint8 Instance,
    uint8 Channel )
```

This function used to get the current output state of the PWM channel.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index

Returns

Pwm_Ftm_OutputStateType FTM_PWM_OUTPUT_STATE_LOW The output pin is in LOW logic level
 FTM_PWM_OUTPUT_STATE_HIGH The output pin is in HIGH logic level

6.1.6.16 Ftm_Pwm_Ip_SetPowerState()

```
void Ftm_Pwm_Ip_SetPowerState (
    uint8 Instance,
    Ftm_Pwm_Ip_PowerStateType PowerState )
```

Put PWM channel based on FTM to target power mode.

This function will disable clock if target power mode is Low power, and enable clock if target power mode is full power

- nPowerState value is either PWM_LOW_POWER or PWM_FULL_POWER
- the values associated to either types are configured by tooling and the physical values associated are written to hardware registry.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>PowerState</i>	Target mode value

Returns

none

6.1.6.17 Ftm_Pwm_Ip_GetChannelState()

```
uint16 Ftm_Pwm_Ip_GetChannelState (
    uint8 Instance,
    uint8 Channel )
```

This function read the duty cycle value from the duty cycle array and returns this value.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index

Returns

uint16 Duty cycle value

6.1.6.18 Ftm_Pwm_Ip_SetPhaseShift()

```
void Ftm_Pwm_Ip_SetPhaseShift (
    uint8 Instance,
    uint8 Channel,
    uint16 Period,
    uint16 PhaseShift,
    boolean SoftwareTrigger )
```

This function set phase shift value and also force duty cycle to 50%.

In order to have Phase-Shifted Full-Bridge controller, Pwm_SetPhaseShift is introduced. This function bases on FTM Combine mode with C(n) and C(n+1) combine to generate leading edge and trailing edge. Pwm_SetPhaseShift allows to set both phase shift value and period, the duty value is fixed to 50%.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index
in	<i>Period</i>	PWM signal period value
in	<i>PhaseShift</i>	Phase shift value
in	<i>SoftwareTrigger</i>	Buffer value be loaded or not

Returns

```
void
```

6.1.6.19 Ftm_Pwm_Ip_SetDutyPhaseShift()

```
void Ftm_Pwm_Ip_SetDutyPhaseShift (
    uint8 Instance,
    uint8 Channel,
    uint16 DutyCycle,
    uint16 PhaseShift,
    boolean SyncUpdate )
```

This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)

Ftm_Pwm_Ip_SetDutyPhaseShift allows to set both phase shift and duty cycle value, The phase shift is the offset of the leading edge of the signal in respect to period starting point.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index
in	<i>DutyCycle</i>	Pwm duty cycle value 0x0000 for 0% ... 0x8000 for 100%
in	<i>PhaseShift</i>	Phase shift value (in ticks)
in	<i>SyncUpdate</i>	TRUE Set the phase shift and duty cycle value base on the synchronization when calling Pwm_SyncUpdate. FALSE Set phase shift and duty cycle value immediately

Returns

void

6.1.6.20 Ftm_Pwm_Ip_UnMaskOutputChannels()

```
void Ftm_Pwm_Ip_UnMaskOutputChannels (
    uint8 Instance,
    uint32 ChannelsMask,
    boolean SoftwareTrigger )
```

This function will unmask the output of the channels and at match events will be ignored by the masked channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>ChannelsMask</i>	The mask which will select which channels will ignore match events.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.

6.1.6.21 Ftm_Pwm_Ip_EnableTrigger()

```
void Ftm_Pwm_Ip_EnableTrigger (
    uint8 Instance,
    uint32 TriggerMask )
```

This function enables trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

<i>Instance</i>	FTM hardware module index TriggerMask Bit mask will be clear for FTM_EXTTRIG.
-----------------	---

Returns

void

6.1.6.22 Ftm_Pwm_Ip_DisableTrigger()

```
void Ftm_Pwm_Ip_DisableTrigger (
    uint8 Instance,
    uint32 TriggerMask )
```

This function disables trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

<i>Instance</i>	FTM hardware module index
<i>TriggerMask</i>	Bit mask will be clear for FTM_EXTTRIG.

Returns

void

6.1.6.23 Ftm_Pwm_Ip_SetChannelDeadTime()

```
void Ftm_Pwm_Ip_SetChannelDeadTime (
    uint8 Instance,
    uint8 Channel,
    uint16 DeadTimeVal )
```

This function is used to update the deadtime at runtime for Pwm channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The FTM hardware channel index
in	<i>DeadTimeVal</i>	Deadtime value

Returns

void

6.2 Pwm Driver

6.2.1 Detailed Description

Data Structures

- struct [Pwm_ChannelConfigType](#)
Pwm channel high level configuration structure. [More...](#)
- struct [Pwm_ConfigType](#)
Pwm high level configuration structure. [More...](#)

Macros

- `#define PWM_DUTY_CYCLE_100`
100% duty cycle
- `#define PWM_E_INIT_FAILED`
API Pwm_Init service called with wrong parameter.
- `#define PWM_E_UNINIT`
API service used without module initialization.
- `#define PWM_E_PARAM_CHANNEL`
API service used with an invalid channel Identifier.
- `#define PWM_E_PERIOD_UNCHANGEABLE`
Usage of unauthorized PWM service on PWM channel configured a fixed period.
- `#define PWM_E_ALREADY_INITIALIZED`
API Pwm_Init service called while the PWM driver has already been initialized.
- `#define PWM_E_PARAM_POINTER`
Generated when a NULL_PTR pointer is passed to Pwm_GetVersionInfo function.
- `#define PWM_E_NOT_DISENGAGED`
Generated when Pwm_SetPowerState is called while the PWM module is still in use.
- `#define PWM_E_POWER_STATE_NOT_SUPPORTED`
The requested power state is not supported by the PWM module.
- `#define PWM_E_TRANSITION_NOT_POSSIBLE`
Generated The requested power state is not reachable from the current one.
- `#define PWM_E_PERIPHERAL_NOT_PREPARED`
Generated when Pwm_SetPowerState has been called without having called the API Pwm_PreparePowerState before.
- `#define PWM_E_PERIODVALUE`
Pwm_SetPeriodAndDuty called with invalid period range.
- `#define PWM_E_PARAM_NOTIFICATION`
Invalid polarity selected for edge notification.
- `#define PWM_E_PARAM_NOTIFICATION_NULL`
NULL_PTR function is configured as notification callback.
- `#define PWM_E_DUTYCYCLE_RANGE`
Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty called with invalid duty cycle range.
- `#define PWM_E_COUNTERBUS`

- Generated when `Pwm_SetCounterBus` is called with an invalid Bus.
- #define `PWM_E_CHANNEL_OFFSET_VALUE`
- Generated when the configured offset for the OPWMB channel is more than the period of the associated MCB channel.
- #define `PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE`
- Generated when the requested offset value plus the current requested duty cycle leads to programming event B over the Period value leading to unexpected behavior of the PWM signal.
- #define `PWM_E_PARAM_INSTANCE`
- Generated when the module id is more than the number of module that supported by this platform.
- #define `PWM_E_OPWMT_CHANNEL_TRIGGER_RANGE`
- Generated when the configured trigger value for the OPWMT channel is equal or greater than the period of the channel.
- #define `PWM_E_SET_CHANNEL_OUTPUT`
- Generated when the output state value for the SetChannelOutput of the channel.
- #define `PWM_E_UNEXPECTED_ISR`
- Generated when an ISR has been triggered.
- #define `PWM_E_PARAM_PHASESHIFT_RANGE`
- Generated when requested phase shift value greater than 0x4000 (50%)
- #define `PWM_E_CHANNEL_PHASE_SHIFT_NOT_SUPPORTED`
- Generated when given channel does not support phase shift feature.
- #define `PWM_E_DUTY_SYNCHRONOUS_NOT_SUPPORTED`
- Generated when given channel does not support duty synchronous feature. (`PHASE_SHIFTED_SYNCED` and `PHASE_SHIFTED_COMPLEMENTARY`)
- #define `PWM_E_TRIGGER_MASK`
- Generated when bit mask is not compatible with hardware register.
- #define `PWM_E_FORCE_OUTPUT_NOT_SUPPORTED`
- Generated when given channel does not support force output to zero feature.
- #define `PWM_E_FORCE_OUT`
- Generated when given channel does not support set force out feature.
- #define `PWM_E_PARAM_CONFIG`
- Generated when the requested resource is configured to be unavailable on the current core.
- #define `PWM_E_DEADTIME_RANGE`
- Generated when the configured dead time value is not valid.
- #define `PWM_E_SETOUTPUTTOIDLE_NOT_SUPPORTED`
- Generated when the configured dead time value is not valid.
- #define `PWM_INIT_ID`
- API service ID of `Pwm_Init` function.
- #define `PWM_DEINIT_ID`
- API service ID of `Pwm_DeInit` function.
- #define `PWM_SETDUTYCYCLE_ID`
- API service ID of `Pwm_SetDutyCycle` function.
- #define `PWM_SETPERIODANDDUTY_ID`
- API service ID of `Pwm_SetPeriodAndDuty` function.
- #define `PWM_SETOUTPUTTOIDLE_ID`
- API service ID of `Pwm_SetOutputToIdle` function.
- #define `PWM_GETOUTPUTSTATE_ID`
- API service ID of `Pwm_GetOutputState` function.
- #define `PWM_DISABLENOTIFICATION_ID`

- API service ID of Pwm_DisableNotification function.*

• #define [PWM_ENABLENOTIFICATION_ID](#)
- API service ID of Pwm_EnableNotification function.*

• #define [PWM_GETVERSIONINFO_ID](#)
- API service ID of Pwm_GetVersionInfo function.*

• #define [PWM_SETPOWERSTATE_ID](#)
- API service ID of Pwm_SetPowerState function.*

• #define [PWM_GETCURRENTPOWERSTATE_ID](#)
- API service ID of Pwm_GetCurrentPowerState function.*

• #define [PWM_GETTARGETPOWERSTATE_ID](#)
- API service ID of Pwm_GetTargetPowerState function.*

• #define [PWM_PREPAREPOWERSTATE_ID](#)
- API service ID of Pwm_PrepPowerState function.*

• #define [PWM_MAIN_POWERTRANSITIONMANAGER_ID](#)
- API service ID of Pwm_Main_PowerTransitionManager function.*

• #define [PWM_GETCHANNELSTATE_ID](#)
- API service ID of Pwm_GetChannelState function.*

• #define [PWM_FORCEOUTPUTTOZERO_ID](#)
- API service ID of Pwm_ForceOutputToZero function.*

• #define [PWM_SETCOUNTERBUS_ID](#)
- API service ID of Pwm_SetCounterBus function.*

• #define [PWM_SETCHANNELOUTPUT_ID](#)
- API service ID of Pwm_SetChannelOutput function.*

• #define [PWM_SETTRIGGERDELAY_ID](#)
- API service ID of Pwm_SetTriggerDelay function.*

• #define [PWM_SETCLOCKMODE_ID](#)
- API service ID of Pwm_SetClockMode function.*

• #define [PWM_SYNCUPDATE_ID](#)
- API service ID of Pwm_SyncUpdate function.*

• #define [PWM_SETPERIODANDDUTY_NO_UPDATE_ID](#)
- API service ID of Pwm_SetPeriodAndDuty_NoUpdate function.*

• #define [PWM_SETDUTYCYCLE_NO_UPDATE_ID](#)
- API service ID of Pwm_SetDutyCycle_NoUpdate function.*

• #define [PWM_SETCHANNELDEADTIME_ID](#)
- API service ID of Pwm_SetChannelDeadTime function.*

• #define [PWM_ENABLETRIGGER_ID](#)
- API service ID of Pwm_EnableTrigger function.*

• #define [PWM_DISABLETRIGGER_ID](#)
- API service ID of Pwm_DisableTrigger function.*

• #define [PWM_RESETCOUNTERENABLE_ID](#)
- API service ID of Pwm_ResetCounterEnable function.*

• #define [PWM_RESETCOUNTERDISABLE_ID](#)
- API service ID of Pwm_ResetCounterDisable function.*

• #define [PWM_MASKOUTPUT_ID](#)
- API service ID of Pwm_MaskOutputs function.*

• #define [PWM_UNMASKOUTPUT_ID](#)
- API service ID of Pwm_UnMaskOutputs function.*

- `#define PWM_DISABLERELOADNOTIF_ID`
API service ID of Pwm_DisableReloadNotification function.
- `#define PWM_ENABLERELOADNOTIF_ID`
API service ID of Pwm_EnableReloadNotification function.
- `#define PWM_SETCHANNELFORCEOUT_ID`
API service ID of Pwm_SetChannelForceOut function.
- `#define PWM_SETDUTYPHASESHIFT_ID`
API service ID of Pwm_SetDutyPhaseShift function.

Types Reference

- `typedef uint8 Pwm_ChannelType`
PWM channel type.
- `typedef uint8 Pwm_InstanceType`
PWM channel type.
- `typedef Pwm_Ipw_PeriodType Pwm_PeriodType`
PWM period type.
- `typedef void(* Pwm_NotifyType) (void)`
Channel notification typedef.

Enum Reference

- `enum Pwm_OutputStateType`
Output signal level.
- `enum Pwm_EdgeNotificationType`
Edge notification type.
- `enum Pwm_ChannelClassType`
PWM channel class type.
- `enum Pwm_PowerStateType`
Power state type.
- `enum Pwm_PowerStateRequestResultType`
Result of power state type.
- `enum Pwm_PrescalerType`
Prescaler type.

Function Reference

- `void Pwm_Init (const Pwm_ConfigType *ConfigPtr)`
This function initializes the Pwm driver.
- `void Pwm_DeInit (void)`
This function deinitializes the Pwm driver.
- `void Pwm_SetDutyCycle (Pwm_ChannelType ChannelNumber, uint16 DutyCycle)`
This function sets the dutycycle for the specified Pwm channel.

- void [Pwm_SetPeriodAndDuty](#) ([Pwm_ChannelType](#) ChannelNumber, [Pwm_PeriodType](#) Period, uint16 DutyCycle)
This function sets the period and the dutycycle for the specified Pwm channel.
- void [Pwm_SetOutputToIdle](#) ([Pwm_ChannelType](#) ChannelNumber)
This function sets the generated pwm signal to the idle value configured.
- [Pwm_OutputStateType](#) [Pwm_GetOutputState](#) ([Pwm_ChannelType](#) ChannelNumber)
This function returns the signal output state.
- void [Pwm_EnableNotification](#) ([Pwm_ChannelType](#) ChannelNumber, [Pwm_EdgeNotificationType](#) Notification)
This function enables the user notifications.
- void [Pwm_DisableNotification](#) ([Pwm_ChannelType](#) ChannelNumber)
This function disables the user notifications.
- void [Pwm_GetVersionInfo](#) ([Std_VersionInfoType](#) *versioninfo)
This function returns Pwm driver version details.
- uint16 [Pwm_GetChannelState](#) ([Pwm_ChannelType](#) ChannelNumber)
This function returns the duty cycle of the channel passed as parameter.
- void [Pwm_ForceOutputToZero](#) ([Pwm_ChannelType](#) ChannelNumber, boolean Force)
This function enables-disables the forcing of the output of a given channel to logic 0.
- void [Pwm_SetClockMode](#) ([Pwm_PrescalerType](#) Prescaler)
Implementation specific function to change the peripheral clock frequency.
- void [Pwm_SetChannelDeadTime](#) ([Pwm_ChannelType](#) ChannelNumber, uint16 DeadTimeTicks)
- void [Pwm_SetDutyCycle_NoUpdate](#) ([Pwm_ChannelType](#) ChannelNumber, uint16 DutyCycle)
This function sets the values of dutycycle for the specified Pwm channel but without updating the PWM output.
- void [Pwm_SetPeriodAndDuty_NoUpdate](#) ([Pwm_ChannelType](#) ChannelNumber, [Pwm_PeriodType](#) Period, uint16 DutyCycle)
This function sets the values of the period and the dutycycle for the specified Pwm channel into the hardware buffers but without updating the PWM output..
- void [Pwm_SyncUpdate](#) (uint8 ModuleId)
Implementation specific function to updates duty synchronization.
- void [Pwm_SetDutyPhaseShift](#) ([Pwm_ChannelType](#) ChannelNumber, uint16 DutyCycle, uint16 PhaseShift, boolean SyncUpdate)
This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)
- void [Pwm_EnableTrigger](#) (uint8 TriggerHostId, uint16 TriggerMask)
This function enable trigger generation for specific source.
- void [Pwm_DisableTrigger](#) (uint8 TriggerHostId, uint16 TriggerMask)
This function disable trigger generation for specific source.
- void [Pwm_ResetCounterEnable](#) (uint8 ModuleId)
This function shall enable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.
- void [Pwm_ResetCounterDisable](#) (uint8 ModuleId)
This function shall disable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.
- void [Pwm_MaskOutputs](#) (uint8 ModuleId, uint8 ChannelMask)
This function force channels output to their inactive state.
- void [Pwm_UnMaskOutputs](#) (uint8 ModuleId, uint8 ChannelMask)
This function puts channels output to normal operation state.
- void [Pwm_EnableReloadNotification](#) (uint8 ModuleId)
This function enables the user reload notifications.
- void [Pwm_DisableReloadNotification](#) (uint8 ModuleId)
This function enables the user reload notifications.

- Std_ReturnType [Pwm_SetPowerState](#) ([Pwm_PowerStateRequestResultType](#) *Result)
Enters the already prepared power state.
- Std_ReturnType [Pwm_GetCurrentPowerState](#) ([Pwm_PowerStateType](#) *CurrentPowerState, [Pwm_PowerStateRequestResultType](#) *Result)
Get the current power state of the Pwm HW unit.
- Std_ReturnType [Pwm_GetTargetPowerState](#) ([Pwm_PowerStateType](#) *TargetPowerState, [Pwm_PowerStateRequestResultType](#) *Result)
Get the target power state of the Pwm HW unit.
- Std_ReturnType [Pwm_PreparePowerState](#) ([Pwm_PowerStateType](#) PowerState, [Pwm_PowerStateRequestResultType](#) *Result)
Starts the needed process to allow the Pwm HW module to enter the requested power state.
- void [Pwm_Main_PowerTransitionManager](#) (void)
Cyclically called and supervises the power state transitions.

6.2.2 Data Structure Documentation

6.2.2.1 struct Pwm_ChannelConfigType

Pwm channel high level configuration structure.

Definition at line 707 of file Pwm.h.

Data Fields

- const [Pwm_ChannelType](#) ChannelId
Id for the logical channel.
- const [Pwm_ChannelClassType](#) PwmChannelClass
Channel class type: Variable/Fixed period.
- const [Pwm_IpwChannelConfigType](#) [IpwChannelCfg](#)
The type of ip channel configured.
- const [Pwm_OutputStateType](#) ChannelIdleState
The state of the channel output in idle mode.

6.2.2.1.1 Field Documentation

6.2.2.1.1.1 ChannelId `const Pwm\_ChannelType ChannelId`

Id for the logical channel.

Definition at line 710 of file Pwm.h.

6.2.2.1.1.2 PwmChannelClass `const Pwm_ChannelClassType PwmChannelClass`

Channel class type: Variable/Fixed period.

Definition at line 712 of file Pwm.h.

6.2.2.1.1.3 IpwChannelCfg `const Pwm_IpwChannelConfigType IpwChannelCfg`

The type of ip channel configured.

Definition at line 714 of file Pwm.h.

6.2.2.1.1.4 ChannelIdleState `const Pwm_OutputStateType ChannelIdleState`

The state of the channel output in idle mode.

Definition at line 716 of file Pwm.h.

6.2.2.2 struct Pwm_ConfigType

Pwm high level configuration structure.

Definition at line 726 of file Pwm.h.

Data Fields

- `const Pwm_ChannelType NumChannels`
Number of Pwm configured channels.
- `const Pwm_ChannelConfigType(* PwmChannelsConfig )[]`
Pointer to the list of Pwm configured channels.
- `const Pwm_InstanceType NumInstances`
Number of Pwm configured instances.
- `const Pwm_IpwInstanceConfigType(* PwmInstancesConfig )[]`
Pointer to the list of Pwm configured channels.
- `const Pwm_ChannelType HwToLogicChannelMap [(12U)]`
Index table to translate HW channels to logical used to process interrupts for notifications.

6.2.2.2.1 Field Documentation

6.2.2.2.1.1 NumChannels `const Pwm_ChannelType NumChannels`

Number of Pwm configured channels.

Definition at line 733 of file Pwm.h.

6.2.2.2.1.2 PwmChannelsConfig `const Pwm_ChannelConfigType (* PwmChannelsConfig) [ ]`

Pointer to the list of Pwm configured channels.

Definition at line 735 of file Pwm.h.

6.2.2.2.1.3 NumInstances `const Pwm_InstanceType NumInstances`

Number of Pwm configured instances.

Definition at line 738 of file Pwm.h.

6.2.2.2.1.4 PwmInstancesConfig `const Pwm_IpwInstanceConfigType (* PwmInstancesConfig) [ ]`

Pointer to the list of Pwm configured channels.

Definition at line 740 of file Pwm.h.

6.2.2.2.1.5 HwToLogicChannelMap `const Pwm_ChannelType HwToLogicChannelMap [ (12U) ]`

Index table to translate HW channels to logical used to process interrupts for notifications.

Definition at line 748 of file Pwm.h.

6.2.3 Macro Definition Documentation**6.2.3.1 PWM_DUTY_CYCLE_100**

```
#define PWM_DUTY_CYCLE_100
```

100% duty cycle

Errors and exceptions that will be detected by the PWM driver generated when `Pwm_SetDutyCycle` or `Pwm_SetPeriodAndDuty` are called with a value for duty cycle out of valid range [0x0000, 0x8000]

Definition at line 144 of file Pwm.h.

6.2.3.2 PWM_E_INIT_FAILED

```
#define PWM_E_INIT_FAILED
```

API Pwm_Init service called with wrong parameter.

Errors and exceptions that will be detected by the PWM driver

Definition at line 151 of file Pwm.h.

6.2.3.3 PWM_E_UNINIT

```
#define PWM_E_UNINIT
```

API service used without module initialization.

Errors and exceptions that will be detected by the PWM driver

Definition at line 157 of file Pwm.h.

6.2.3.4 PWM_E_PARAM_CHANNEL

```
#define PWM_E_PARAM_CHANNEL
```

API service used with an invalid channel Identifier.

Errors and exceptions that will be detected by the PWM driver

Definition at line 163 of file Pwm.h.

6.2.3.5 PWM_E_PERIOD_UNCHANGEABLE

```
#define PWM_E_PERIOD_UNCHANGEABLE
```

Usage of unauthorized PWM service on PWM channel configured a fixed period.

Errors and exceptions that will be detected by the PWM driver

Definition at line 169 of file Pwm.h.

6.2.3.6 PWM_E_ALREADY_INITIALIZED

```
#define PWM_E_ALREADY_INITIALIZED
```

API Pwm_Init service called while the PWM driver has already been initialized.

Errors and exceptions that will be detected by the PWM driver

Definition at line 175 of file Pwm.h.

6.2.3.7 PWM_E_PARAM_POINTER

```
#define PWM_E_PARAM_POINTER
```

Generated when a NULL_PTR pointer is passed to Pwm_GetVersionInfo function.

Errors and exceptions that will be detected by the PWM driver

Definition at line 181 of file Pwm.h.

6.2.3.8 PWM_E_NOT_DISENGAGED

```
#define PWM_E_NOT_DISENGAGED
```

Generated when Pwm_SetPowerState is called while the PWM module is still in use.

Errors and exceptions that will be detected by the PWM driver

Definition at line 187 of file Pwm.h.

6.2.3.9 PWM_E_POWER_STATE_NOT_SUPPORTED

```
#define PWM_E_POWER_STATE_NOT_SUPPORTED
```

The requested power state is not supported by the PWM module.

Errors and exceptions that will be detected by the PWM driver

Definition at line 193 of file Pwm.h.

6.2.3.10 PWM_E_TRANSITION_NOT_POSSIBLE

```
#define PWM_E_TRANSITION_NOT_POSSIBLE
```

Generated The requested power state is not reachable from the current one.

Errors and exceptions that will be detected by the PWM driver

Definition at line 199 of file Pwm.h.

6.2.3.11 PWM_E_PERIPHERAL_NOT_PREPARED

```
#define PWM_E_PERIPHERAL_NOT_PREPARED
```

Generated when Pwm_SetPowerState has been called without having called the API Pwm_PreparePowerState before.

Errors and exceptions that will be detected by the PWM driver

Definition at line 206 of file Pwm.h.

6.2.3.12 PWM_E_PERIODVALUE

```
#define PWM_E_PERIODVALUE
```

Pwm_SetPeriodAndDuty called with invalid period range.

Generated when Pwm_SetPeriodAndDuty is called with a value for period out of valid range [0x0000, PWM_MAX_PERIOD]

Definition at line 213 of file Pwm.h.

6.2.3.13 PWM_E_PARAM_NOTIFICATION

```
#define PWM_E_PARAM_NOTIFICATION
```

Invalid polarity selected for edge notification.

Will be generated when an invalid polarity, edge notification is requested for one PWM channel. Due to the limitations that are present in the eMIOS implementation not all the polarity notifications combinations can be supported.

Definition at line 221 of file Pwm.h.

6.2.3.14 PWM_E_PARAM_NOTIFICATION_NULL

```
#define PWM_E_PARAM_NOTIFICATION_NULL
```

NULL_PTR function is configured as notification callback.

Will be generated when a NULL_PTR function is configured as notification callback for one PWM channel and Pwm_EnableNotification is called for that channel

Definition at line 228 of file Pwm.h.

6.2.3.15 PWM_E_DUTYCYCLE_RANGE

```
#define PWM_E_DUTYCYCLE_RANGE
```

Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty called with invalid duty cycle range.

Generated when Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty are called with a value for duty cycle out of valid range [0x0000, 0x8000]

Definition at line 235 of file Pwm.h.

6.2.3.16 PWM_E_COUNTERBUS

```
#define PWM_E_COUNTERBUS
```

Generated when Pwm_SetCounterBus is called with an invalid Bus.

Errors and exceptions that will be detected by the PWM driver

Definition at line 241 of file Pwm.h.

6.2.3.17 PWM_E_CHANNEL_OFFSET_VALUE

```
#define PWM_E_CHANNEL_OFFSET_VALUE
```

Generated when the configured offset for the OPWMB channel is more than the period of the associated MCB channel.

Errors and exceptions that will be detected by the PWM driver

Definition at line 248 of file Pwm.h.

6.2.3.18 PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE

```
#define PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE
```

Generated when the requested offset value plus the current requested duty cycle leads to programming event B over the Period value leading to unexpected behavior of the PWM signal.

Errors and exceptions that will be detected by the PWM driver

Definition at line 255 of file Pwm.h.

6.2.3.19 PWM_E_PARAM_INSTANCE

```
#define PWM_E_PARAM_INSTANCE
```

Generated when the module id is more than the number of module that supported by this platform.

Errors and exceptions that will be detected by the PWM driver

Definition at line 262 of file Pwm.h.

6.2.3.20 PWM_E_OPWMT_CHANNEL_TRIGGER_RANGE

```
#define PWM_E_OPWMT_CHANNEL_TRIGGER_RANGE
```

Generated when the configured trigger value for the OPWMT channel is equal or greater than the period of the channel.

Errors and exceptions that will be detected by the PWM driver

Definition at line 269 of file Pwm.h.

6.2.3.21 PWM_E_SET_CHANNEL_OUTPUT

```
#define PWM_E_SET_CHANNEL_OUTPUT
```

Generated when the output state value for the SetChannelOutput of the channel.

Errors and exceptions that will be detected by the PWM driver

Definition at line 276 of file Pwm.h.

6.2.3.22 PWM_E_UNEXPECTED_ISR

```
#define PWM_E_UNEXPECTED_ISR
```

Generated when an ISR has been triggered.

1. when the driver is not initialized
2. for a HW channel that is not used by any logic channel
3. for a logic channel that has no notification configured

Errors and exceptions that will be detected by the PWM driver

Definition at line 285 of file Pwm.h.

6.2.3.23 PWM_E_PARAM_PHASESHIFT_RANGE

```
#define PWM_E_PARAM_PHASESHIFT_RANGE
```

Generated when requested phase shift value greater than 0x4000 (50%)

Pwm_SetPhaseShift only works with Combine channel (COMBINED_SYNCED or COMBINED_COMPLEMENTARY) Which do not support matching at next cycle. The duty cycle is always fixed value at 50%, so Phase Shift cannot greater than 50%

Definition at line 293 of file Pwm.h.

6.2.3.24 PWM_E_CHANNEL_PHASE_SHIFT_NOT_SUPPORTED

```
#define PWM_E_CHANNEL_PHASE_SHIFT_NOT_SUPPORTED
```

Generated when given channel does not support phase shift feature.

For FTM, only combine mode (COMBINED_SYNCED and COMBINED_COMPLEMENTARY) is supported. For eMIOS, only OPWMB, OPWM and OPWMT mode is supported.

Definition at line 300 of file Pwm.h.

6.2.3.25 PWM_E_DUTY_SYNCHRONOUS_NOT_SUPPORTED

```
#define PWM_E_DUTY_SYNCHRONOUS_NOT_SUPPORTED
```

Generated when given channel does not support duty synchronous feature. (PHASE_SHIFTED_SYNCED and PHASE_SHIFTED_COMPLEMENTARY)

For FTM, please note that Modified Combine mode does not support synchronous update. Therefore Pwm_SetDutyCycle_NoUpdate and Pwm_SetPeriodAndDuty_NoUpdate should not be called in this case. For eMIOS, please note that channels using DAOC mode or channels in idle state does not support.

Definition at line 311 of file Pwm.h.

6.2.3.26 PWM_E_TRIGGER_MASK

```
#define PWM_E_TRIGGER_MASK
```

Generated when bit mask is not compatible with hardware register.

Definition at line 317 of file Pwm.h.

6.2.3.27 PWM_E_FORCE_OUTPUT_NOT_SUPPORTED

```
#define PWM_E_FORCE_OUTPUT_NOT_SUPPORTED
```

Generated when given channel does not support force output to zero feature.

Only channels in FlexPWM and FTM module are supported.

Definition at line 323 of file Pwm.h.

6.2.3.28 PWM_E_FORCE_OUT

```
#define PWM_E_FORCE_OUT
```

Generated when given channel does not support set force out feature.

Only channels in FlexPWM is supported.

Definition at line 329 of file Pwm.h.

6.2.3.29 PWM_E_PARAM_CONFIG

```
#define PWM_E_PARAM_CONFIG
```

Generated when the requested resource is configured to be unavailable on the current core.

Only multi-core configuration is available.

Definition at line 335 of file Pwm.h.

6.2.3.30 PWM_E_DEADTIME_RANGE

```
#define PWM_E_DEADTIME_RANGE
```

Generated when the configured dead time value is not valid.

Errors and exceptions that will be detected by the PWM driver

Definition at line 341 of file Pwm.h.

6.2.3.31 PWM_E_SETOUTPUTTOIDLE_NOT_SUPPORTED

```
#define PWM_E_SETOUTPUTTOIDLE_NOT_SUPPORTED
```

Generated when the configured dead time value is not valid.

Errors and exceptions that will be detected by the PWM driver

Definition at line 347 of file Pwm.h.

6.2.3.32 PWM_INIT_ID

```
#define PWM_INIT_ID
```

API service ID of Pwm_Init function.

Parameters used when raising an error/exception

Definition at line 353 of file Pwm.h.

6.2.3.33 PWM_DEINIT_ID

```
#define PWM_DEINIT_ID
```

API service ID of Pwm_DeInit function.

Parameters used when raising an error/exception

Definition at line 359 of file Pwm.h.

6.2.3.34 PWM_SETDUTYCYCLE_ID

```
#define PWM_SETDUTYCYCLE_ID
```

API service ID of Pwm_SetDutyCycle function.

Parameters used when raising an error/exception

Definition at line 365 of file Pwm.h.

6.2.3.35 PWM_SETPERIODANDDUTY_ID

```
#define PWM_SETPERIODANDDUTY_ID
```

API service ID of Pwm_SetPeriodAndDuty function.

Parameters used when raising an error/exception

Definition at line 371 of file Pwm.h.

6.2.3.36 PWM_SETOUTPUTTOIDLE_ID

```
#define PWM_SETOUTPUTTOIDLE_ID
```

API service ID of Pwm_SetOutputToIdle function.

Parameters used when raising an error/exception

Definition at line 377 of file Pwm.h.

6.2.3.37 PWM_GETOUTPUTSTATE_ID

```
#define PWM_GETOUTPUTSTATE_ID
```

API service ID of Pwm_GetOutputState function.

Parameters used when raising an error/exception

Definition at line 383 of file Pwm.h.

6.2.3.38 PWM_DISABLENOTIFICATION_ID

```
#define PWM_DISABLENOTIFICATION_ID
```

API service ID of Pwm_DisableNotification function.

Parameters used when raising an error/exception

Definition at line 389 of file Pwm.h.

6.2.3.39 PWM_ENABLENOTIFICATION_ID

```
#define PWM_ENABLENOTIFICATION_ID
```

API service ID of Pwm_EnableNotification function.

Parameters used when raising an error/exception

Definition at line 395 of file Pwm.h.

6.2.3.40 PWM_GETVERSIONINFO_ID

```
#define PWM_GETVERSIONINFO_ID
```

API service ID of Pwm_GetVersionInfo function.

Parameters used when raising an error/exception

Definition at line 401 of file Pwm.h.

6.2.3.41 PWM_SETPOWERSTATE_ID

```
#define PWM_SETPOWERSTATE_ID
```

API service ID of Pwm_SetPowerState function.

Parameters used when raising an error/exception

Definition at line 407 of file Pwm.h.

6.2.3.42 PWM_GETCURRENTPOWERSTATE_ID

```
#define PWM_GETCURRENTPOWERSTATE_ID
```

API service ID of Pwm_GetCurrentPowerState function.

Parameters used when raising an error/exception

Definition at line 413 of file Pwm.h.

6.2.3.43 PWM_GETTARGETPOWERSTATE_ID

```
#define PWM_GETTARGETPOWERSTATE_ID
```

API service ID of Pwm_GetTargetPowerState function.

Parameters used when raising an error/exception

Definition at line 419 of file Pwm.h.

6.2.3.44 PWM_PREPAREPOWERSTATE_ID

```
#define PWM_PREPAREPOWERSTATE_ID
```

API service ID of Pwm_PrepPowerState function.

Parameters used when raising an error/exception

Definition at line 425 of file Pwm.h.

6.2.3.45 PWM_MAIN_POWERTRANSITIONMANAGER_ID

```
#define PWM_MAIN_POWERTRANSITIONMANAGER_ID
```

API service ID of Pwm_Main_PowerTransitionManager function.

Parameters used when raising an error/exception

Definition at line 431 of file Pwm.h.

6.2.3.46 PWM_GETCHANNELSTATE_ID

```
#define PWM_GETCHANNELSTATE_ID
```

API service ID of Pwm_GetChannelState function.

Parameters used when raising an error/exception

Definition at line 437 of file Pwm.h.

6.2.3.47 PWM_FORCEOUTPUTTOZERO_ID

```
#define PWM_FORCEOUTPUTTOZERO_ID
```

API service ID of Pwm_ForceOutputToZero function.

Parameters used when raising an error/exception

Definition at line 443 of file Pwm.h.

6.2.3.48 PWM_SETCOUNTERBUS_ID

```
#define PWM_SETCOUNTERBUS_ID
```

API service ID of Pwm_SetCounterBus function.

Parameters used when raising an error/exception

Definition at line 449 of file Pwm.h.

6.2.3.49 PWM_SETCHANNELOUTPUT_ID

```
#define PWM_SETCHANNELOUTPUT_ID
```

API service ID of Pwm_SetChannelOutput function.

Parameters used when raising an error/exception

Definition at line 455 of file Pwm.h.

6.2.3.50 PWM_SETTRIGGERDELAY_ID

```
#define PWM_SETTRIGGERDELAY_ID
```

API service ID of Pwm_SetTriggerDelay function.

Parameters used when raising an error/exception

Definition at line 461 of file Pwm.h.

6.2.3.51 PWM_SETCLOCKMODE_ID

```
#define PWM_SETCLOCKMODE_ID
```

API service ID of Pwm_SetClockMode function.

Parameters used when raising an error/exception

Definition at line 467 of file Pwm.h.

6.2.3.52 PWM_SYNCUPDATE_ID

```
#define PWM_SYNCUPDATE_ID
```

API service ID of Pwm_SyncUpdate function.

Parameters used when raising an error/exception

Definition at line 473 of file Pwm.h.

6.2.3.53 PWM_SETPERIODANDDUTY_NO_UPDATE_ID

```
#define PWM_SETPERIODANDDUTY_NO_UPDATE_ID
```

API service ID of Pwm_SetPeriodAndDuty_NoUpdate function.

Parameters used when raising an error/exception

Definition at line 479 of file Pwm.h.

6.2.3.54 PWM_SETDUTYCYCLE_NO_UPDATE_ID

```
#define PWM_SETDUTYCYCLE_NO_UPDATE_ID
```

API service ID of Pwm_SetDutyCycle_NoUpdate function.

Parameters used when raising an error/exception

Definition at line 485 of file Pwm.h.

6.2.3.55 PWM_SETCHANNELDEADTIME_ID

```
#define PWM_SETCHANNELDEADTIME_ID
```

API service ID of Pwm_SetChannelDeadTime function.

Parameters used when raising an error/exception

Definition at line 491 of file Pwm.h.

6.2.3.56 PWM_ENABLETRIGGER_ID

```
#define PWM_ENABLETRIGGER_ID
```

API service ID of Pwm_EnableTrigger function.

Parameters used when raising an error/exception

Definition at line 497 of file Pwm.h.

6.2.3.57 PWM_DISABLETRIGGER_ID

```
#define PWM_DISABLETRIGGER_ID
```

API service ID of Pwm_DisableTrigger function.

Parameters used when raising an error/exception

Definition at line 503 of file Pwm.h.

6.2.3.58 PWM_RESETCOUNTERENABLE_ID

```
#define PWM_RESETCOUNTERENABLE_ID
```

API service ID of Pwm_ResetCounterEnable function.

Parameters used when raising an error/exception

Definition at line 509 of file Pwm.h.

6.2.3.59 PWM_RESETCOUNTERDISABLE_ID

```
#define PWM_RESETCOUNTERDISABLE_ID
```

API service ID of Pwm_ResetCounterDisable function.

Parameters used when raising an error/exception

Definition at line 515 of file Pwm.h.

6.2.3.60 PWM_MASKOUTPUT_ID

```
#define PWM_MASKOUTPUT_ID
```

API service ID of Pwm_MaskOutputs function.

Parameters used when raising an error/exception

Definition at line 521 of file Pwm.h.

6.2.3.61 PWM_UNMASKOUTPUT_ID

```
#define PWM_UNMASKOUTPUT_ID
```

API service ID of Pwm_UnMaskOutputs function.

Parameters used when raising an error/exception

Definition at line 527 of file Pwm.h.

6.2.3.62 PWM_DISABLERELOADNOTIF_ID

```
#define PWM_DISABLERELOADNOTIF_ID
```

API service ID of Pwm_DisableReloadNotification function.

Parameters used when raising an error/exception

Definition at line 533 of file Pwm.h.

6.2.3.63 PWM_ENABLERELOADNOTIF_ID

```
#define PWM_ENABLERELOADNOTIF_ID
```

API service ID of Pwm_EnableReloadNotification function.

Parameters used when raising an error/exception

Definition at line 539 of file Pwm.h.

6.2.3.64 PWM_SETCHANNELFORCEOUT_ID

```
#define PWM_SETCHANNELFORCEOUT_ID
```

API service ID of Pwm_SetChannelForceOut function.

Parameters used when raising an error/exception

Definition at line 545 of file Pwm.h.

6.2.3.65 PWM_SETDUTYPHASESHIFT_ID

```
#define PWM_SETDUTYPHASESHIFT_ID
```

API service ID of Pwm_SetDutyPhaseShift function.

Parameters used when raising an error/exception

Definition at line 551 of file Pwm.h.

6.2.4 Types Reference

6.2.4.1 Pwm_ChannelType

```
typedef uint8 Pwm_ChannelType
```

PWM channel type.

Definition at line 681 of file Pwm.h.

6.2.4.2 Pwm_InstanceType

```
typedef uint8 Pwm_InstanceType
```

PWM channel type.

Definition at line 687 of file Pwm.h.

6.2.4.3 Pwm_PeriodType

```
typedef Pwm_Ipw_PeriodType Pwm_PeriodType
```

PWM period type.

Definition at line 693 of file Pwm.h.

6.2.4.4 Pwm_NotifyType

```
typedef void(* Pwm_NotifyType) (void)
```

Channel notification typedef.

Definition at line 700 of file Pwm.h.

6.2.5 Enum Reference

6.2.5.1 Pwm_OutputStateType

```
enum Pwm_OutputStateType
```

Output signal level.

This enumeration specifies the return type of Pwm_GetOutputState

Enumerator

PWM_LOW	PWM level is logic low.
PWM_HIGH	PWM level is logic high.

Definition at line 562 of file Pwm.h.

6.2.5.2 Pwm_EdgeNotificationType

```
enum Pwm_EdgeNotificationType
```

Edge notification type.

This enumeration defines the type of edge transition that can generate a notification

Enumerator

PWM_FALLING_EDGE	A notification will be generated on the falling edge.
PWM_RISING_EDGE	A notification will be generated on the rising edge.
PWM_BOTH_EDGES	A notification will be generated on any state transition.

Definition at line 576 of file Pwm.h.

6.2.5.3 Pwm_ChannelClassType

```
enum Pwm_ChannelClassType
```

PWM channel class type.

This field will specify what parameters can be altered for the selected channel

Enumerator

PWM_VARIABLE_PERIOD	The period and duty cycle can be altered.
PWM_FIXED_PERIOD	Only the duty cycle can be altered.
PWM_FIXED_PERIOD_SHIFTED	Only the duty cycle can be altered.

Definition at line 592 of file Pwm.h.

6.2.5.4 Pwm_PowerStateType

enum `Pwm_PowerStateType`

Power state type.

Power state currently active or set as target power state.

Enumerator

PWM_FULL_POWER	PWM full power mode.
PWM_LOW_POWER	PWM low power mode.
PWM_NODEFINE_POWER	PWM no define power mode.

Definition at line 608 of file Pwm.h.

6.2.5.5 Pwm_PowerStateRequestResultType

enum `Pwm_PowerStateRequestResultType`

Result of power state type.

Result of the requests related to power state transitions.

Enumerator

PWM_SERVICE_ACCEPTED	Power state change executed.
PWM_NOT_INIT	Module not initialized.
PWM_SEQUENCE_ERROR	Wrong API call sequence.
PWM_HW_FAILURE	The HW module has a failure which prevents it to enter the required power state.
PWM_POWER_STATE_NOT_SUPP	Module does not support the requested power state.
PWM_TRANS_NOT_POSSIBLE	Module cannot transition directly from the current power state to the requested power state.

Definition at line 624 of file Pwm.h.

6.2.5.6 Pwm_PrescalerType

enum `Pwm_PrescalerType`

Prescaler type.

This enumeration specifies the possible types of prescalers used to configure base-clock timers

Enumerator

PWM_PRIMARY_PRESCALER	Selected value is the default/primary prescaler.
PWM_ALTERNATIVE_PRESCALER	Selected value is the alternative configured prescaler.

Definition at line 645 of file Pwm.h.

6.2.6 Function Reference

6.2.6.1 Pwm_Init()

```
void Pwm_Init (
    const Pwm_ConfigType * ConfigPtr )
```

This function initializes the Pwm driver.

The function Pwm_Init shall initialize all internal variables and the used PWM structure of the microcontroller according to the parameters specified in configPtr. If the duty cycle parameter equals:

- 0% or 100% : Then the PWM output signal shall be in the state according to the configured polarity parameter;
- >0% and <100%: Then the PWM output signal shall be modulated according to parameters period, duty cycle and configured polarity.

The function Pwm_SetDutyCycle shall update the duty cycle always at the end of the period if supported by the implementation and configured with PwmDutycycleUpdatedEndperiod.

The driver shall avoid spikes on the PWM output signal when updating the PWM period and duty.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error PWM_E_PERIOD_UNCHANGEABLE if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The function `Pwm_Init` shall disable all notifications. The reason is that the users of these notifications may not be ready. They can call `Pwm_EnableNotification` to start notifications.

The function `Pwm_Init` shall only initialize the configured resources and shall not touch resources that are not configured in the configuration file.

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

If development error detection is enabled, calling the routine `Pwm_Init` while the PWM driver and hardware are already initialized will cause a development error `PWM_E_ALREADY_INITIALIZED`. The desired functionality shall be left without any action.

For pre-compile and link time configuration variants, a `NULL` pointer shall be passed to the initialization routine. In this case the check for this `NULL` pointer has to be omitted.

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Parameters

in	<i>ConfigPtr</i>	Pointer to PWM top configuration structure
----	------------------	--

Returns

`void`

6.2.6.2 Pwm_DeInit()

```
void Pwm_DeInit (  
    void )
```

This function deinitializes the Pwm driver.

The function `Pwm_DeInit` shall deinitialize the PWM module.

The function `Pwm_DeInit` shall set the state of the PWM output signals to the idle state.

The function `Pwm_DeInit` shall disable PWM interrupts and PWM signal edge notifications.

The function `Pwm_DeInit` shall be pre-compile time configurable On-Off by the configuration parameter `PwmDeInitApi` function prototype.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function `Pwm_GetOutputState`.

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Returns

`void`

6.2.6.3 Pwm_SetDutyCycle()

```
void Pwm_SetDutyCycle (
    Pwm_ChannelType ChannelNumber,
    uint16 DutyCycle )
```

This function sets the dutycycle for the specified Pwm channel.

The function Pwm_SetDutyCycle shall set the duty cycle of the PWM channel.

The function Pwm_SetDutyCycle shall set the PWM output state according to the configured polarity parameter, when the duty cycle = 0% or 100%. The function Pwm_SetDutyCycle shall modulate the PWM output signal according to parameters period, duty cycle and configured polarity, when the duty cycle > 0 % and < 100%.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value.

As an implementation guide, the following source code example is given:

```
AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;
```

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.2.6.4 Pwm_SetPeriodAndDuty()

```
void Pwm_SetPeriodAndDuty (
    Pwm_ChannelType ChannelNumber,
```

```
Pwm_PeriodType Period,
uint16 DutyCycle )
```

This function sets the period and the dutycycle for the specified Pwm channel.

The function Pwm_SetPeriodAndDuty shall set the duty cycle of the PWM channel.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error PWM_E_PERIOD_UNCHANGEABLE if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value.

As an implementation guide, the following source code example is given:

```
AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;
```

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>Period</i>	Pwm signal period value
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.2.6.5 Pwm_SetOutputToIdle()

```
void Pwm_SetOutputToIdle (
    Pwm_ChannelType ChannelNumber )
```

This function sets the generated pwm signal to the idle value configured.

The function `Pwm_SetOutputToIdle` shall set immediately the PWM output to the configured Idle state.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PARAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

After the call of the function `Pwm_SetOutputToIdle`, variable period type channels shall be reactivated either using the Api `Pwm_SetPeriodAndDuty()` to activate the PWM channel with the new passed period or Api `Pwm_SetDutyCycle()` to activate the PWM channel with the old period.

After the call of the function `Pwm_SetOutputToIdle`, fixed period type channels shall be reactivated using only the API `Pwm_SetDutyCycle()` to activate the PWM channel with the old period.

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
----	----------------------	-------------------------------------

Returns

void

6.2.6.6 Pwm_GetOutputState()

```
Pwm_OutputStateType Pwm_GetOutputState (
    Pwm_ChannelType ChannelNumber )
```

This function returns the signal output state.

The function `Pwm_GetOutputState` shall read the internal state of the PWM output signal and return it as defined in the diagram below (see `PWM_SWS`).

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PARAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return `pwm` level low for the function `Pwm_GetOutputState`.

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

Due to real time constraint and setting of the PWM channel (project dependant), the output state can be modified just after the call of the service `Pwm_GetOutputState`.

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
----	----------------------	-------------------------------------

Returns

`Pwm_OutputStateType` Pwm signal output logic value

Return values

<i>PWM_LOW</i>	The output state of PWM channel is low
<i>PWM_HIGH</i>	The output state of PWM channel is high

6.2.6.7 Pwm_EnableNotification()

```
void Pwm_EnableNotification (
    Pwm_ChannelType ChannelNumber,
    Pwm_EdgeNotificationType Notification )
```

This function enables the user notifications.

The function `Pwm_EnableNotification` shall enable the PWM signal edge notification according to notification parameter. If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PA↔RAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return `pwm` level low for the function `Pwm_GetOutputState`.

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>Notification</i>	Notification type to be enabled

Returns

`void`

6.2.6.8 Pwm_DisableNotification()

```
void Pwm_DisableNotification (
    Pwm_ChannelType ChannelNumber )
```

This function disables the user notifications.

If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PA↔RAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

- Return pwm level low for the function Pwm_GetOutputState.

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

All functions from the PWM module except Pwm_Init, Pwm_DeInit and Pwm_GetVersionInfo shall be re-entrant for different PWM channel numbers. In order to keep a simple module implementation, no check of PWM088 must be performed by the module. The function Pwm_DisableNotification shall be pre compile time configurable On-Off by the configuration parameter: PwmNotificationSupported.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
----	----------------------	-------------------------------------

Returns

void

6.2.6.9 Pwm_GetVersionInfo()

```
void Pwm_GetVersionInfo (
    Std_VersionInfoType * versioninfo )
```

This function returns Pwm driver version details.

The function Pwm_GetVersionInfo shall return the version information of this module. The version information includes: Module Id, Vendor Id, Vendor specific version number.

Parameters

out	<i>versioninfo</i>	Pointer to Std_VersionInfoType output variable
-----	--------------------	--

Returns

void

6.2.6.10 Pwm_GetChannelState()

```
uint16 Pwm_GetChannelState (
    Pwm_ChannelType ChannelNumber )
```

This function returns the duty cycle of the channel passed as parameter.

The function `Pwm_GetChannelState` shall return the `DutyCycle` of the channel. In case the channel is idle, the returned value will be zero.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
----	----------------------	-------------------------------------

Returns

uint16 DutyCycle of the requested channel

6.2.6.11 Pwm_ForceOutputToZero()

```
void Pwm_ForceOutputToZero (
    Pwm_ChannelType ChannelNumber,
    boolean Force )
```

This function enables-disables the forcing of the output of a given channel to logic 0.

The function `Pwm_GetChannelState` shall return the `DutyCycle` of the channel. In case the channel is idle, the returned value will be zero.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>Force</i>	Boolean value to state if the output of the channel will be forced to zero logic or not

Returns

void

6.2.6.12 Pwm_SetClockMode()

```
void Pwm_SetClockMode (
    Pwm_PrescalerType Prescaler )
```

Implementation specific function to change the peripheral clock frequency.

This function is useful to set the prescalers that divide the PWM channels clock frequency.

Parameters

in	<i>Prescaler</i>	Prescaler type
----	------------------	----------------

Returns

void

6.2.6.13 Pwm_SetChannelDeadTime()

```
void Pwm_SetChannelDeadTime (
    Pwm_ChannelType ChannelNumber,
    uint16 DeadTimeTicks )
```

This function is used to update the deadtime at runtime for Pwm channels.

Parameters

in	<i>ChannelNumber</i>	Pwm channel id
in	<i>DeadTimeTicks</i>	Dead Time value in ticks

Returns

void

6.2.6.14 Pwm_SetDutyCycle_NoUpdate()

```
void Pwm_SetDutyCycle_NoUpdate (
    Pwm_ChannelType ChannelNumber,
    uint16 DutyCycle )
```

This function sets the values of dutycycle for the specified Pwm channel but without updating the PWM output.

The function Pwm_SetDutyCycle_NoUpdate shall set the duty cycle of the PWM channel to the coresponding hardware buffers without updating the wave form on the output pin. This feature will allow a pre-buffering of new PWM duty cycle values for several channel, which can all be updated synchronos by calling Pwm_SyncUpdate.

The function Pwm_SetDutyCycle_NoUpdate shall set the PWM output state according to the configured polarity parameter, when the duty cycle = 0% or 100%. The function Pwm_SetDutyCycle_NoUpdate shall modulate the PWM output signal according to parameters period, duty cycle and configured polarity, when the duty cycle > 0 % and < 100%.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PARAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function `Pwm_GetOutputState`.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value. As an implementation guide, the following source code example is given: `AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;`

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.2.6.15 Pwm_SetPeriodAndDuty_NoUpdate()

```
void Pwm_SetPeriodAndDuty_NoUpdate (
    Pwm_ChannelType ChannelNumber,
    Pwm_PeriodType Period,
    uint16 DutyCycle )
```

This function sets the values of the period and the dutycycle for the specified Pwm channel into the hardware buffers but without updating the PWM output..

The function `Pwm_SetPeriodAndDuty_NoUpdate` shall set the period and duty cycle of the PWM channel to the corresponding hardware buffers without updating the wave form on the output pin. This feature will allow a pre-buffering of new PWM duty cycle values for several channel, which can all be updated synchronously by calling `Pwm_SyncUpdate`.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error `PWM_E_PERIOD_UNCHANGEABLE` if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PARAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function `Pwm_GetOutputState`.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value. As an implementation guide, the following source code example is given: `AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;`

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>Period</i>	Pwm signal period value
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.2.6.16 Pwm_SyncUpdate()

```
void Pwm_SyncUpdate (
    uint8 ModuleId )
```

Implementation specific function to updates duty synchronization.

This function is used to update duty synchronization for channels in given module, this should be called after calling [Pwm_SetPeriodAndDuty_NoUpdate\(\)](#) or [Pwm_SetDutyCycle_NoUpdate\(\)](#) API.

Parameters

in	<i>ModuleId</i>	pwm module id(instance ID) Ex : PWM_EMIOS_INSTANCE_0 PWM_EMIOS_INSTANCE_1 ... PWM_FTM_INSTANCE_0
----	-----------------	---

Returns

void

6.2.6.17 Pwm_SetDutyPhaseShift()

```
void Pwm_SetDutyPhaseShift (
    Pwm_ChannelType ChannelNumber,
    uint16 DutyCycle,
    uint16 PhaseShift,
    boolean SyncUpdate )
```

This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)

Pwm_SetDutyPhaseShift allows to set both phase shift and duty cycle value, The phase shift is the offset of the leading edge of the signal in respect to period starting point.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>DutyCycle</i>	Pwm duty cycle value 0x0000 for 0% ... 0x8000 for 100%
in	<i>PhaseShift</i>	Phase shift value (in ticks)
in	<i>SyncUpdate</i>	Update duty and phases shift value synchronization for channels in given module or not TRUE Set the phase shift and duty cycle value base on the synchronization when calling Pwm_SyncUpdate. FALSE Set phase shift and duty cycle value immediately

Returns

void

6.2.6.18 Pwm_EnableTrigger()

```
void Pwm_EnableTrigger (
    uint8 TriggerHostId,
    uint16 TriggerMask )
```

This function enable trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

in	<i>TriggerHostId</i>	Pwm hardware module id
in	<i>TriggerMask</i>	Bit mask will be set to enable trigger with corresponding sources.

Returns

void

6.2.6.19 Pwm_DisableTrigger()

```
void Pwm_DisableTrigger (
    uint8 TriggerHostId,
    uint16 TriggerMask )
```

This function disable trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

in	<i>TriggerHostId</i>	Pwm hardware module id
in	<i>TriggerMask</i>	Bit mask will be cleared to disable trigger with corresponding sources.

Returns

void

6.2.6.20 Pwm_ResetCounterEnable()

```
void Pwm_ResetCounterEnable (
    uint8 ModuleId )
```

This function shall enable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Returns

void

6.2.6.21 Pwm_ResetCounterDisable()

```
void Pwm_ResetCounterDisable (
    uint8 ModuleId )
```

This function shall disable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Returns

void

6.2.6.22 Pwm_MaskOutputs()

```
void Pwm_MaskOutputs (
    uint8 ModuleId,
    uint8 ChannelMask )
```

This function force channels output to their inactive state.

Corresponding bits with channel will be masked: Bit 0 Channel 0 Output Mask Bit 1 Channel 1 Output Mask Bit 2 Channel 2 Output Mask Bit 3 Channel 3 Output Mask Bit 4 Channel 4 Output Mask Bit 5 Channel 5 Output Mask

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
in	<i>ChannelMask</i>	Bit mask will be set to mask corresponding channel.

Returns

void

6.2.6.23 Pwm_UnMaskOutputs()

```
void Pwm_UnMaskOutputs (
    uint8 ModuleId,
    uint8 ChannelMask )
```

This function puts channels output to normal operation state.

Corresponding bits with channel will be masked: Bit 0 Channel 0 Output Mask Bit 1 Channel 1 Output Mask Bit 2 Channel 2 Output Mask Bit 3 Channel 3 Output Mask Bit 4 Channel 4 Output Mask Bit 5 Channel 5 Output Mask

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
in	<i>ChannelMask</i>	Bit mask will be cleared to unmask corresponding channel.

Returns

void

6.2.6.24 Pwm_EnableReloadNotification()

```
void Pwm_EnableReloadNotification (
    uint8 ModuleId )
```

This function enables the user reload notifications.

The function Pwm_EnableReloadNotification shall enable the PWM reload notification If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ModuleId and raise development error PWM_E_PARAM_↔ INSTANCE if the parameter ModuleId is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Returns

void

6.2.6.25 Pwm_DisableReloadNotification()

```
void Pwm_DisableReloadNotification (
    uint8 ModuleId )
```

This function enables the user reload notifications.

The function Pwm_DisableReloadNotification shall disable the PWM reload notification If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ModuleId and raise development error PWM_E_PARAM_INVALID_INSTANCE if the parameter ModuleId is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Returns

void

6.2.6.26 Pwm_SetPowerState()

```
Std_ReturnType Pwm_SetPowerState (  
    Pwm_PowerStateRequestResultType * Result )
```

Enters the already prepared power state.

This API configures the Pwm module so that it enters the already prepared power state, chosen between a predefined set of configured ones.

Parameters

out	<i>Result</i>	Pointer to a variable to store the result of this function
-----	---------------	--

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Power Mode changed.
<i>E_NOT_OK</i>	Request rejected.

6.2.6.27 Pwm_GetCurrentPowerState()

```
Std_ReturnType Pwm_GetCurrentPowerState (  
    Pwm_PowerStateType * CurrentPowerState,  
    Pwm_PowerStateRequestResultType * Result )
```

Get the current power state of the Pwm HW unit.

This API returns the current power state of the Pwm HW unit.

Parameters

out	<i>CurrentPowerState</i>	The current power mode of the Pwm HW Unit is returned in this parameter
out	<i>Result</i>	Pointer to a variable to store the result of this function

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Mode could be read.
<i>E_NOT_OK</i>	Service is rejected.

6.2.6.28 Pwm_GetTargetPowerState()

```
Std_ReturnType Pwm_GetTargetPowerState (
    Pwm_PowerStateType * TargetPowerState,
    Pwm_PowerStateRequestResultType * Result )
```

Get the target power state of the Pwm HW unit.

This API returns the target power state of the Pwm HW unit.

Parameters

out	<i>TargetPowerState</i>	The Target power mode of the Pwm HW Unit is returned in this parameter.
out	<i>Result</i>	Pointer to a variable to store the result of this function.

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Mode could be read.
<i>E_NOT_OK</i>	Service is rejected.

6.2.6.29 Pwm_PreparePowerState()

```
Std_ReturnType Pwm_PreparePowerState (
    Pwm_PowerStateType PowerState,
    Pwm_PowerStateRequestResultType * Result )
```

Starts the needed process to allow the Pwm HW module to enter the requested power state.

This API starts the needed process to allow the Pwm HW module to enter the requested power state.

Parameters

in	<i>PowerState</i>	The target power state intended to be attained.
out	<i>Result</i>	Pointer to a variable to store the result of this function.

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Mode could be read.
<i>E_NOT_OK</i>	Service is rejected.

6.2.6.30 Pwm_Main_PowerTransitionManager()

```
void Pwm_Main_PowerTransitionManager (
    void )
```

Cyclically called and supervises the power state transitions.

This API is cyclically called and supervises the power state transitions, checking for the readiness of the module and issuing the callbacks.

Parameters

in	<i>none</i>	
out	<i>none</i>	

Returns

none

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

